

课程须知

- 本课程适用软件版本: MWORKS.Syslab2024a
- 本课程示例运行需要软件首选项加载:
 - 基础库(TyBase)
 - 数学库(TyMath)
 - 图形库(TyPlot)
 - 控制系统库(TyControlSystems)

MWORKS.Syslab控制系统工具箱

鲍丙瑞 张万宽

苏州同元软控信息技术有限公司

2024-01-07



TONGYUAN
Software and Control

目录

1. 控制系统工具箱功能概述

2. 线性控制系统的数学模型

3. 线性控制系统分析

4. PID控制器设计

5. 状态反馈控制律设计

01

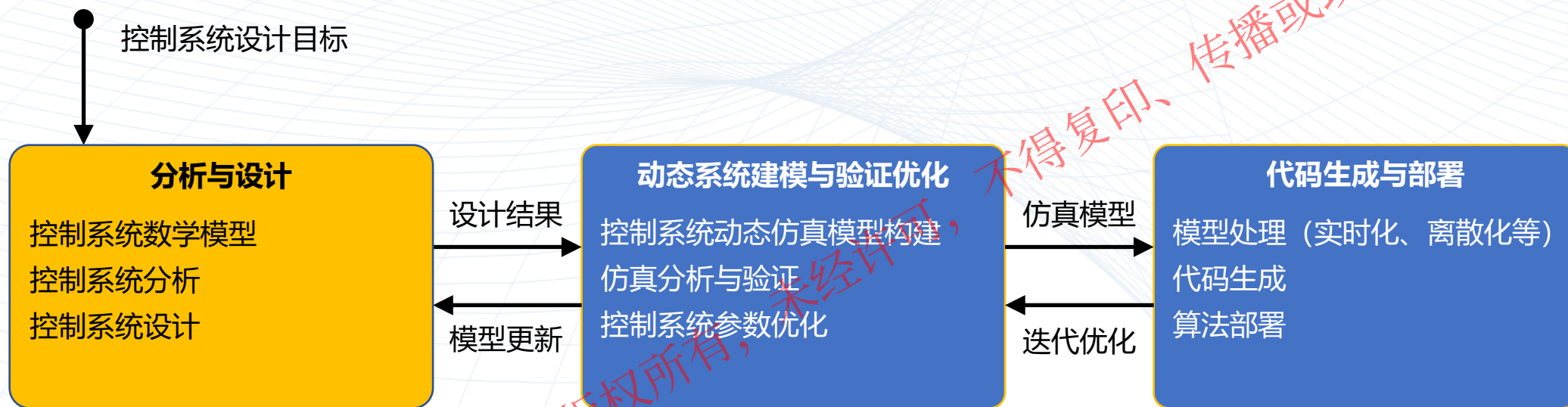
控制系统工具箱功能概述

未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究

1. 控制系统工具箱功能概述

控制系统工具箱在控制系统基于模型的设计流程中的定位



MWORKS控制系统工具箱旨在为控制系统**分析与设计**阶段提供计算支撑

1. 控制系统工具箱功能概述

控制系统工具箱面向**经典控制**和**现代控制**领域，为分析、设计和调节线性控制系统提供算法、模型和应用程序

两类场景

两个工具

四个应用

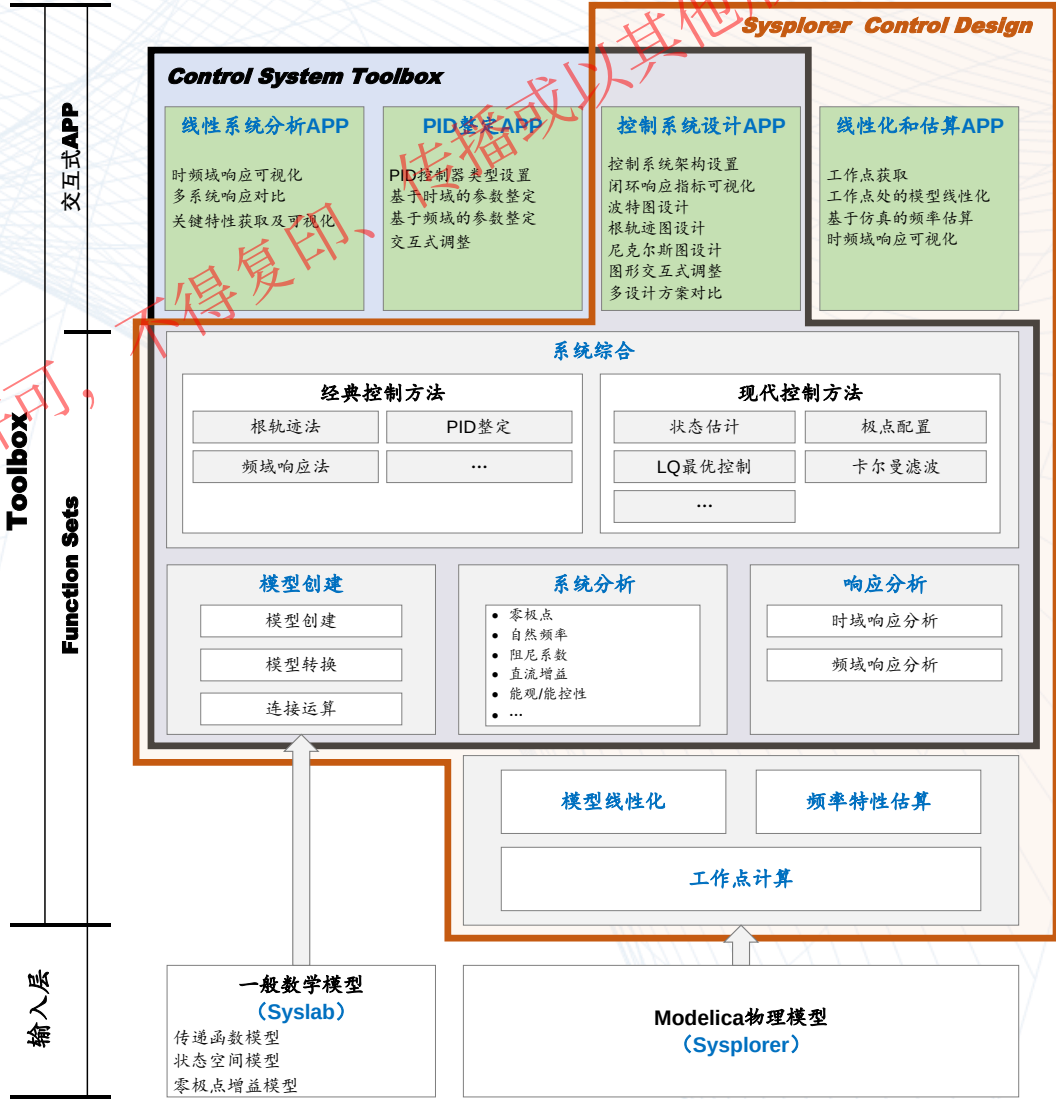
六类功能

- 基于数学模型的控制系统分析、设计与验证
- 基于物理模型的控制系统分析、设计与验证

- 控制系统工具（Control System Toolbox）
- 基于模型的设计工具（Sysplorer Control Design）

- 线性系统分析APP
- 控制系统设计APP
- 线性化与估算APP
- PID整定APP（2023Q3发布）

- 线性模型创建
- 线性系统分析
- PID控制
- 状态估计与最优控制
- 补偿器设计
- 线性化及频率特性估算



1. 控制系统工具箱功能概述

1 线性模型创建

- 线性时不变LTI模型创建
- 模型转换
- 模型连接运算
- 模型简化

2 线性系统分析

- 时域分析与可视化
- 频域分析与可视化
- 根轨迹分析
- 能观能控性分析

3 PID控制器设计

- 各类型连续/离散PID控制器创建
- 基于目标的PID参数计算

4 状态估计与最优控制

- 状态反馈控制律设计
- 卡尔曼滤波器设计
- 线性二次型调节器设计

5 线性化及频率特性估算

- 物理模型线性化
- 物理模型频率特性估算

6 补偿器设计

- 设计目标设定
- 基于根轨迹法的补偿器设计
- 基于频域法的补偿器设计

02

线性控制系统的数学模型

未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究

2.1 线性控制系统的数学模型 - 模型创建

大部分控制系统分析与设计的方法都需要假设系统的模型已知，而获得数学模型有两种方法：

- 从已知的物理规律出发，用数学推导的方式建立系统的数学模型
- 由实验数据拟合系统的数学模型（系统辨识）

线性系统一般可以用以下三种模型进行表示：

1. 传递函数模型
2. 状态空间模型
3. 零极点增益模型

一. 传递函数模型 (tf)

连续时间动态系统一般以微分方程描述，而 LTI 系统则以定常系数线性常微分方程描述。假设系统的输入信号为 $u(t)$ ，输出为 $y(t)$ ，则 n 阶系统的微分方程为：

$$a_n \frac{d^n y(t)}{dt^n} + a_{n-1} \frac{d^{n-1} y(t)}{dt^{n-1}} + \dots + a_1 \frac{dy(t)}{dt} + a_0 y(t) = b_m \frac{d^m u(t)}{dt^m} + b_{m-1} \frac{d^{m-1} u(t)}{dt^{m-1}} + \dots + b_1 \frac{du(t)}{dt} + b_0 u(t) \quad (n \geq m)$$

系统输入量与输出量的 Laplace 变换之比即为其传递函数：

$$G(s) = \frac{Y(s)}{U(s)} = \frac{b_m s^m + b_{m-1} s^{m-1} + \dots + b_1 s + b_0}{a_n s^n + a_{n-1} s^{n-1} + \dots + a_1 s + a_0}, n \geq m$$

传递函数可以表示成两个多项式的比值，在 Syslab 中，多项式可以用向量表示。将多项式的系数按 s 的降幂次序表示得到一个数值向量，分别用 num 与 den 表示分子、分母多项式，再利用 `tf()` 函数即可创建系统传递函数

```
num = [bm, ... b1, b0]
den = [an, ... a1, a0]
# 通过指定分子分母多项式系数创建传递函数
G = tf(num, den)
```

2.1 线性控制系统的数学模型 - 模型创建

一. 传递函数模型 (tf)

示例2.1: Syslab中建立传递函数模型:

$$G(s) = \frac{12s^3 + 24s^2 + 12s + 20}{2s^4 + 4s^3 + 6s^2 + 2s + 2}$$

方式一

```
num = [12, 24, 12, 20]
```

```
den = [2, 4, 6, 2, 2]
```

```
G = tf(num, den)
```

方式二

```
G = tf([12, 24, 12, 20], [2, 4, 6, 2, 2])
```

输出:

$$12s^3 + 24s^2 + 12s + 20$$
$$2s^4 + 4s^3 + 6s^2 + 2s + 2$$

连续时间传递函数模型

考虑如果传递函数分子分母多项式给出的不是完全展开的形式, 而是若干因式的乘积, 或者包括其他运算。这种情况可以定义Laplace算子: `s = tf('s')`, 然后用类似数学表达式的形式直接输入

示例2.2: Syslab中建立传递函数模型:

$$G(s) = \frac{3(s^2 + 3)}{(s + 2)^3 (s^2 + 2s + 1)(s^2 + 5)}$$

```
s = tf('s') # 定义Laplace算子
```

```
G = 3*(s^2+3)/((s+2)^3*(s^2+2*s+1)*(s^2+5))
```

输出:

$$3s^2 + 9$$
$$s^7 + 8s^6 + 30s^5 + 78s^4 + 153s^3 + 198s^2 + 140s + 40$$

连续时间传递函数模型

2.1 线性控制系统的数学模型 - 模型创建

一. 传递函数模型 (tf)

离散时间动态系统一般以差分方程描述，LTI 系统则以定系数线性差分方程描述，对于离散SISO，设定采样周期为 T ，系统的输入为 $u(i)$ ，输出为 $y(i)$ ，则相应差分方程为：

$$a_n y(i+n) + a_{n-1} y(i+n-1) + \cdots + a_1 y(i+1) + a_0 y(i) = b_m u(i+m) + b_{m-1} u(i+m-1) + \cdots + b_1 u(i+1) + b_0 u(i)$$

对上述方程进行 z 变换，得到离散系统的传递函数：

$$H(z) = \frac{Y(z)}{U(z)} = \frac{b_m z^m + b_{m-1} z^{m-1} + \cdots + b_1 z + b_0}{a_n z^n + a_{n-1} z^{n-1} + \cdots + a_1 z + a_0}, n \geq m$$

在 Syslab 中，同样使用 `tf()` 函数创建离散系统传递函数，与连续传递函数不同的是，同时需要指定采样时间 T

示例2.3： Syslab中建立离散系统传递函数模型，其采样周期为 $T = 0.1s$

$$H(z) = \frac{6z^2 - 0.6z - 0.12}{z^4 - z^3 + 0.25z^2 + 0.25z - 0.125}$$

```
num = [bm, ... b1,b0]
den = [an, ... a1,a0]
# 通过指定分子分母多项式系数、采样时间
G = tf(num,den,ts)
```

```
num = [6 -0.6 -0.12]
den = [1 -1 0.25 0.25 -0.125]
H = tf(num,den,0.1)
```

输出：

```
6.0z^2 - 0.6z - 0.12
-----
1.0z^4 - 1.0z^3 + 0.25z^2 + 0.25z - 0.125

Sample Time: 0.1 (seconds)
离散时间传递函数模型
```

Tips:

针对离散时间系统模型，同样可以通过 `z = tf('z')` 定义 z 算子，再进行数学表达式形式创建模型

2.1 线性控制系统的数学模型 - 模型创建

一. 传递函数模型 (tf)

多变量系统模型的一种表述形式为：传递函数矩阵。这是单变量系统传递函数在多变量系统中的直接扩展，一般可写为：

$$G(s) = \begin{bmatrix} G_{11}(s) & G_{12}(s) & \cdots & G_{1n}(s) \\ G_{21}(s) & G_{22}(s) & \cdots & G_{2n}(s) \\ \vdots & \vdots & \ddots & \vdots \\ G_{m1}(s) & G_{m2}(s) & \cdots & G_{mn}(s) \end{bmatrix}$$

其中 $G_{i,j}(s)$ 表示第 i 路输入信号对第 j 路输出信号的放大倍数

方式一：通过创建的子传递函数构造MIMO系统传递函数矩阵

方式二：通过分子、分母向量构成的矩阵构造MIMO系统传递函数矩阵

示例2.4：构建以下MIMO系统的传递函数矩阵

$$G(s) = \begin{bmatrix} \frac{s-1}{s+1} & \frac{100}{(s+4)(s+10.625)} \\ \frac{s+2}{s^2+4s+5} & \frac{s+2}{s^3+4s+20} \end{bmatrix}$$

```
s = tf('s')
G11 = tf([1,-1],[1,1])
G12 = 100/((s+4)*(s+10.625))
G21 = tf([1,2],[1,4,5])
G22 = tf([1,2],[1,0,4,20])
# 通过矩阵命令构造传递函数矩阵
G = [G11 G12; G21 G22]
```

输出

```
输入 1 到输出 1
1.0s - 1.0
-----
1.0s + 1.0

输入 1 到输出 2
1.0s + 2.0
-----
1.0s^2 + 4.0s + 5.0

输入 2 到输出 1
100.0
-----
1.0s^2 + 14.625s + 42.5

输入 2 到输出 2
1.0s + 2.0
-----
1.0s^3 + 4.0s + 20.0

连续时间传递函数模型
```

示例2.5：构建以下离散MIMO系统的传递函数矩阵，采样时间为 $ts = 0.2s$

$$H(z) = \begin{bmatrix} \frac{1}{z+0.3} & \frac{z}{z+0.3} \\ \frac{-z+2}{z+0.3} & \frac{3}{z+0.3} \end{bmatrix}$$

```
nums = [[[1]] [[1,0]]; [[-1,2]] [[3]]]
dens = [[[1,0.3]] [[1,0.3]]; [[1,0.3]]
        [[1,0.3]]]
ts = 0.2
H = tf(nums,dens,ts)
```

输出

```
输入 1 到输出 1
1.0
-----
1.0z + 0.3

输入 1 到输出 2
-1.0z + 2.0
-----
1.0z + 0.3

输入 2 到输出 1
1.0z
-----
1.0z + 0.3

输入 2 到输出 2
3.0
-----
1.0z + 0.3

Sample Time: 0.2
(seconds)
离散时间传递函数模型
```

2.1 线性控制系统的数学模型 - 模型创建

二. 状态空间模型 (ss)

状态空间模型可以描述更广的一类控制系统，包括非线性系统、MIMO系统。针对连续时间 LTI 系统可以描述为：

□ 针对**连续时间** LTI 系统可以描述为：

$$\begin{cases} \dot{x}(t) = Ax(t) + Bu(t) \\ y(t) = Cx(t) + Du(t) \end{cases}$$

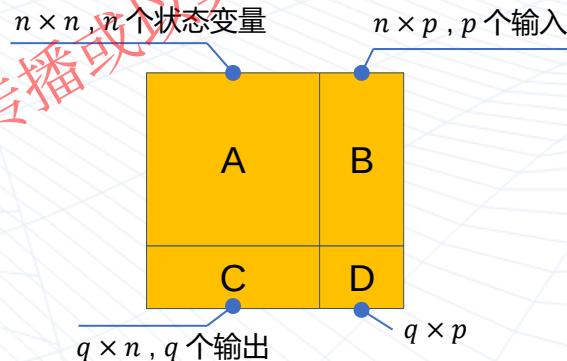
□ 针对**离散时间** LTI 系统可以描述为：

$$\begin{cases} x[n+1] = Ax[n] + Bu[n] \\ y[n] = Cx[n] + Du[n] \end{cases}$$

示例2.6：构建下面的双输入双输出系统状态空间模型

$$\begin{cases} \dot{x}(t) = \begin{bmatrix} -12 & -17.2 & -16.8 & -11.9 \\ 6 & 8.5 & 9.5 & 8 \\ 5 & 8.7 & 3.5 & 6 \\ -6 & -6.5 & -9.7 & -5 \end{bmatrix} x(t) + \begin{bmatrix} 2.5 & 0.1 \\ 2 & 0.5 \\ 3 & 1 \\ 0 & 0.5 \end{bmatrix} u(t) \\ y(t) = \begin{bmatrix} 2 & 3 & 0.8 & 0 \\ 0.5 & 0.56 & 0.3 & -1 \end{bmatrix} x(t) \end{cases}$$

```
A = [-12 -17.2 -16.8 -11.9; 6 8.5 9.5 8; 5 8.7 3.5 6; -6 -6.5 -9.7 -5]
B = [2.5 0.1; 2 0.5; 3 1; 0 0.5]
C = [2 3 0.8 0; 0.5 0.56 0.3 -1]
D = zeros(2,2)
G = ss(A,B,C,D)
```



输出

```
A =
-12.0 -17.2 -16.8 -11.9
  6.0   8.5   9.5   8.0
  5.0   8.7   3.5   6.0
 -6.0  -6.5  -9.7  -5.0

B =
 2.5  0.1
 2.0  0.5
 3.0  1.0
 0.0  0.5

C =
 2.0  3.0  0.8  0.0
 0.5  0.56 0.3 -1.0

D =
 0.0  0.0
 0.0  0.0
```

连续时间状态空间模型

针对离散时间状态空间模型的创建，同样使用 ss 函数， $G = ss(A, B, C, D, ts)$ ，增加ts指定采样时间即可

2.1 线性控制系统的数学模型 - 模型创建

三. 零极点增益模型 (zpk)

零极点增益模型实际上是传递函数的一种特殊形式，其将系统表示为零点 (Zeros)、极点 (Poles) 和增益 (Gain) 相乘的形式：

$$G(s) = k \frac{\prod_{i=1}^m (s + z_i)}{\prod_{j=1}^n (s + p_j)} = k \frac{(s + z_1)(s + z_2) \cdots (s + z_m)}{(s + p_1)(s + p_2) \cdots (s + p_n)}$$

其中：k 为系统增益， $-z_i, i = 1, 2, \dots, m$ 为系统零点， $-p_j, j = 1, 2, \dots, n$ 为系统极点

示例2.7：构建下面的零极点增益模型

$$G(s) = \frac{-2s}{(s-1-i)(s-1+i)(s-2)}$$

```
z = [0]
p = [1-1im, 1+1im, 2]
k = -2
G = zpk(z, p, k)
```

输出

$$\frac{-2s}{(s^2 - 2s + 2)(s - 2)}$$

连续时间传递函数模型

示例2.8：构建离散时间零极点增益模型，采样时间为 $ts = 0.125s$

$$H(z) = \frac{\frac{z}{z-0.3} \cdot 2(z+0.5)}{(z-0.1+i)(z-0.1-i)}$$

```
z = [[[0]]; [[-0.5]]]
p = [[[0.3]]; [[0.1-1im, 0.1+1im]]]
k = [[1]; [2]]
G = zpk(z, p, k, 0.125)
```

输出

输入 1 到输出 1

$$\frac{1.0z}{1.0z - 0.3}$$

输入 1 到输出 2

$$\frac{2.0(1.0z + 0.5)}{1.0z^2 - 0.2z + 1.01}$$

Sample Time: 0.125 (seconds)

离散时间传递函数模型

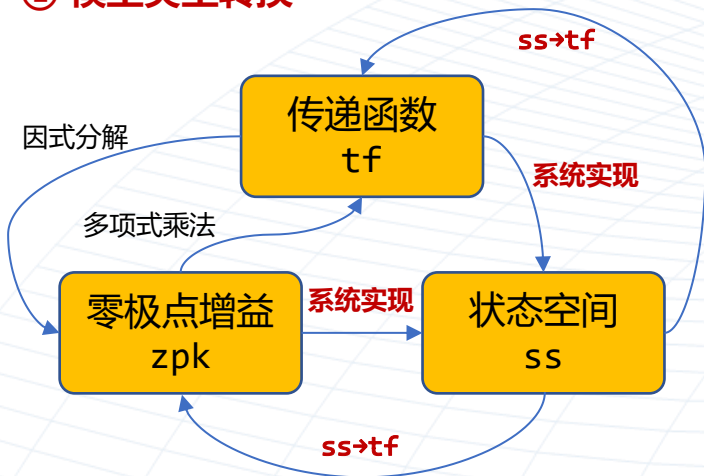
2.1 线性控制系统的数学模型 - 模型创建

模型创建相关主要Syslab函数，其余函数请参阅Syslab-控制系统工具箱帮助文档

函数及调用方式	说明
<code>sys = tf(num,den)</code>	返回变量为sys的连续时间传递函数
<code>sys = tf(num,den,ts)</code>	返回变量为sys的离散时间传递函数，其中ts表示为系统采样时间
<code>s = tf('s')</code>	定义Laplace算子，支持以有理多项式形式创建连续时间传递函数
<code>z = tf('z',ts)</code>	定义Z变换算子及其采样时间ts，支持以有理多项式形式创建离散时间传递函数
<code>sys = zpk(z,p,k)</code>	返回变量为sys的连续系统零极点增益模型
<code>sys = zpk(z,p,k,ts)</code>	返回变量为sys的离散系统零极点增益模型，其中ts表示为系统采样时间
<code>sys = ss(A,B,C,D)</code>	返回变量为sys的连续系统传递函数模型
<code>sys = ss(A,B,C,D,ts)</code>	返回变量为sys的离散系统传递函数模型，其中ts表示为系统采样时间

2.2 线性控制系统的数学模型 - 模型转换

① 模型类型转换



系统实现: 对以下形式的传递函数转化为状态空间表达式, 即为系统实现

$$G(s) = \frac{Y(s)}{U(s)} = \frac{b_m s^m + b_{m-1} s^{m-1} + \dots + b_1 s + b_0}{a_n s^n + a_{n-1} s^{n-1} + \dots + a_1 s + a_0}, n \geq m$$

ss→tf: 状态空间模型到传递函数的转化

$$G(s) = C[sI - A]^{-1}B + D$$

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \vdots \\ \dot{x}_{n-1} \\ \dot{x}_n \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 \\ -\frac{a_0}{a_n} & -\frac{a_1}{a_n} & -\frac{a_2}{a_n} & \dots & -\frac{a_{n-1}}{a_n} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{n-1} \\ x_n \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix} \cdot u$$
$$y = \begin{bmatrix} \frac{b_0 - a_0 b_n}{a_n} & \frac{b_1 - a_1 b_n}{a_n} & \frac{b_2 - a_2 b_n}{a_n} & \dots & \frac{b_{n-1} - a_{n-1} b_n}{a_n} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{n-1} \\ x_n \end{bmatrix} + \frac{b_n}{a_n} \cdot u$$

Syslab中可以直接通过 `tf()`、`ss()`、`zpk()` 函数实现不同表达形式模型之间的相互转换

示例2.9: 将以下状态空间模型转化为传递函数模型

$$\dot{x} = \begin{bmatrix} -2 & -1 \\ 1 & -2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 1 & 1 \\ 2 & -1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$
$$y = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

```
A = [-2 -1; 1 -2]
B = [1 1; 2 -1]
C = [1 0]
D = [0 1]
ltiSys = ss(A, B, C, D);
# 直接使用tf函数进行转换
sys = tf(ltiSys)
```

```
输入 1 到输出 1
      1.0s
-----
1.0s^2 + 4.0s + 5.0

输入 2 到输出 1
      1.0s^2 + 5.0s + 8.0
-----
1.0s^2 + 4.0s + 5.0

连续时间传递函数模型
```

示例2.10: 将以下零极点增益模型转化为状态空间模型

$$H(s) = \frac{10(s+5)}{(s+8)(s+10)(s+2)}$$

```
z = [-5]
p = [-8, -10, -2]
H = zpk(z, p, 10);
# 直接使用ss函数进行转换
sys = ss(H)
```

```
A =
  0.0    4.0    0.0
  0.0    0.0    8.0
 -5.0  -14.5  -20.0
B =
  0.0
  0.0
  2.0
C =
  0.78125  0.625  0.0
D =
  0.0

连续时间状态空间模型
```

2.2 线性控制系统的数学模型 - 模型转换

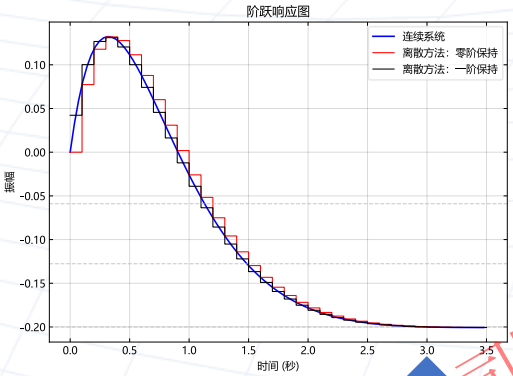
② 连续 - 离散转换

支持连续-离散系统模型的相互转换，以及离散系统之间的重采样

示例2.11: 将以下连续模型离散化，采样时间为 $t_s = 0.1s$

$$G(s) = \frac{s-1}{s^2+4s+5}$$

```
G = tf([1,-1],[1,4,5])
# 默认离散方法: 零阶保持
Gd = c2d(G,0.1)
# 选用一阶保持离散方法
Gd1 = c2d(G,0.1,:foh)
```



输出

```
julia> Gd
0.07735946566180907z - 0.08556727104741424
-----
1.0z^2 - 1.6292810191076135z + 0.6703200460356392

Sample Time: 0.1 (seconds)
离散时间传递函数模型

julia> Gd1
0.04226338595954682z^2 - 0.010930071156678323z - 0.03954112018847362
-----
1.0z^2 - 1.6292810191076135z + 0.6703200460356392

Sample Time: 0.1 (seconds)
离散时间传递函数模型
```

示例2.12: 将以下离散时间系统进行重采样，重采样时间为 $t_s = 0.02s$

$$H(z) = \frac{z-1}{z^2+4z+5}, t_s = 0.1s$$

```
H = tf([1,-1],[1,4,5],0.1)
# 对原系统进行重采样
Hc = d2d(H,0.02)
```

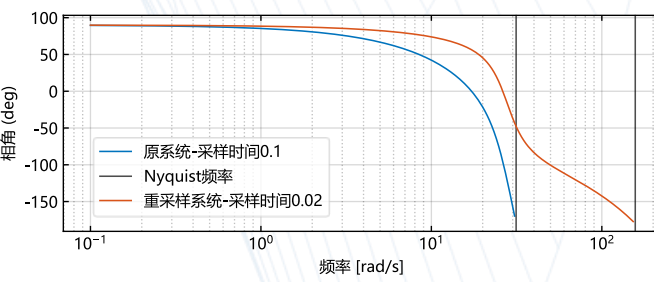
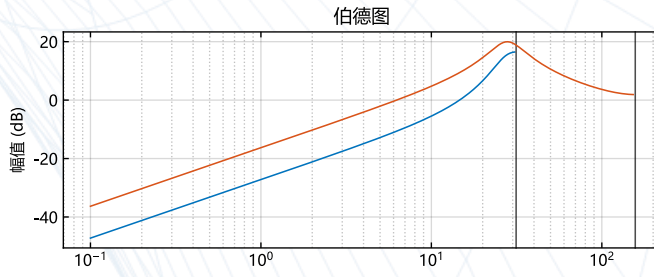
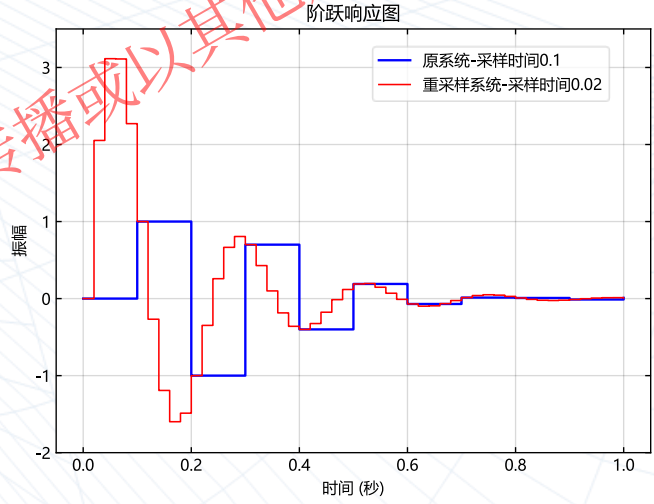
```
julia> H
1.0z - 1.0
-----
1.0z^2 + 1.0z + 0.3

Sample Time: 0.1 (seconds)
离散时间传递函数模型

julia> Hc
2.052778221587413z - 2.0527782215874133
-----
1.0z^2 - 1.5169814293463761z + 0.7860030855966226

Sample Time: 0.02 (seconds)
离散时间传递函数模型
```

输出



2.2 线性控制系统的数学模型 - 模型转换

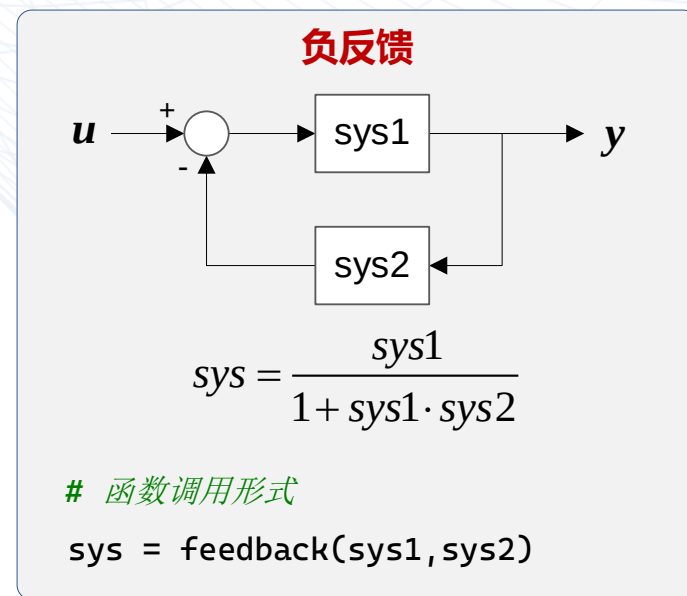
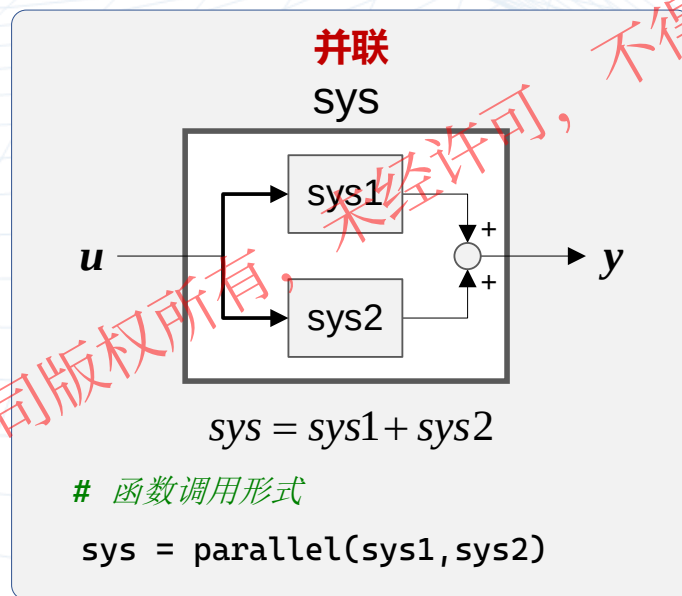
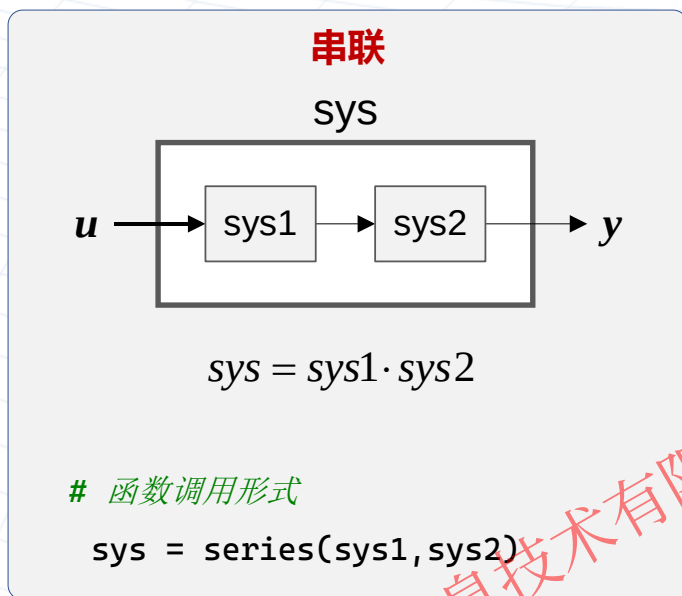
模型转换相关主要Syslab函数，其余函数请参阅Syslab-控制系统工具箱帮助文档

函数及调用方式	说明
<code>sys = tf(ltisys)</code>	将ltisys系统转化为传递函数模型，ltisys可以为ss、zpk
<code>sys = ss(ltiSys)</code>	将ltisys系统转化为状态空间模型，ltisys可以为tf、zpk
<code>sys = zpk(ltiSys)</code>	将ltisys系统转化为零极点增益模型，ltisys可以为tf、ss
<code>sysd = c2d(sysc, Ts)</code> <code>sysd = c2d(sysc, Ts, method)</code>	将连续时间模型转化为离散时间模型
<code>sysc = d2c(sysd)</code> <code>sysc = d2c(sysd, method)</code>	将离散时间模型转化为连续时间模型
<code>sys1 = d2d(sysc, Ts)</code> <code>sys1 = d2d(sysc, Ts, method)</code>	对离散时间动态系统模型 sys 进行重新采样，以产生具有新采样时间 Ts（以秒为单位）的等效离散时间模型 sys1

2.3 线性控制系统的数学模型 – 系统连接与化简

① 系统连接

- 在实际应用中，整个控制系统由被控对象和控制装置组成，存在多个环节组合而成。每个单一模型都可以用一组微分方程或传递函数来描述
- 模型间连接主要有串联连接、并联连接、串并联连接和反馈连接等。对系统的不同连接情况，可以进行模型的化简



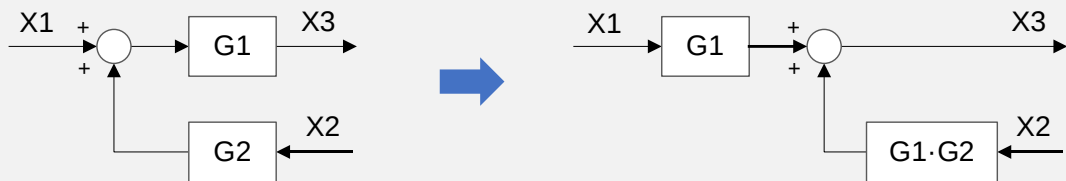
除上述三种典型连接外，Syslab还提供 `append()`、`connect()`、`lft()` 等函数支持一些相对复杂的连接，具体可以参考帮助文档

2.3 线性控制系统的数学模型 – 系统连接与化简

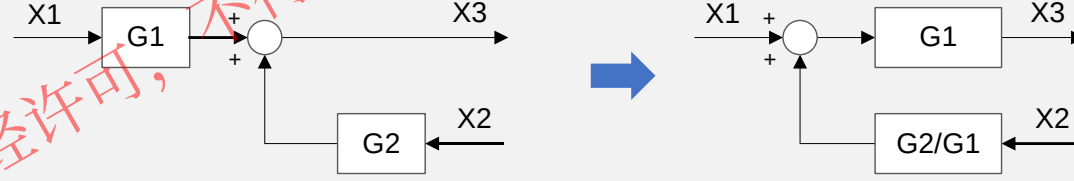
② 系统变换化简

在复杂结构框图化简中，经常需要将某个支路的输入点从一个节点移动到另一个节点上，进而便于系统推导与简化

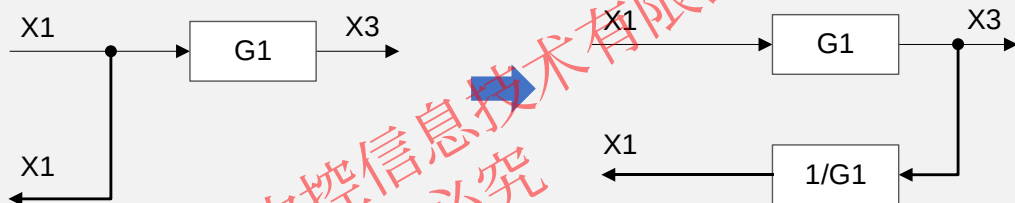
A. 相加点后移等效变换



B. 相加点前移等效变换



C. 分支点后移等效变换

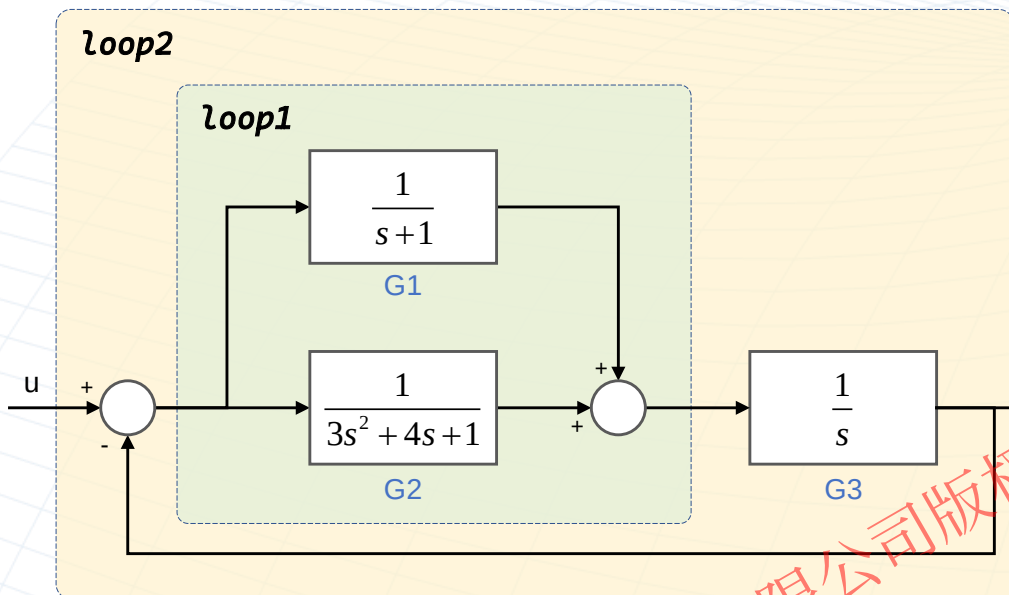


D. 分支点前移等效变换



2.3 线性控制系统的数学模型 – 系统连接与化简

示例2.13: 化简如图系统，求系统的传递函数



```
# 定义各模块模型
G1 = tf([1],[1,1])
G2 = tf([1],[3,4,1])
G3 = tf([1],[1,0])
# 计算loop1
loop1 = parallel(G1,G2)
# 计算loop2
loop2 = feedback(loop1*G3,1)
```

```
julia> loop1
      3s^2 + 5s + 2
      -----
      3s^3 + 7s^2 + 5s + 1
```

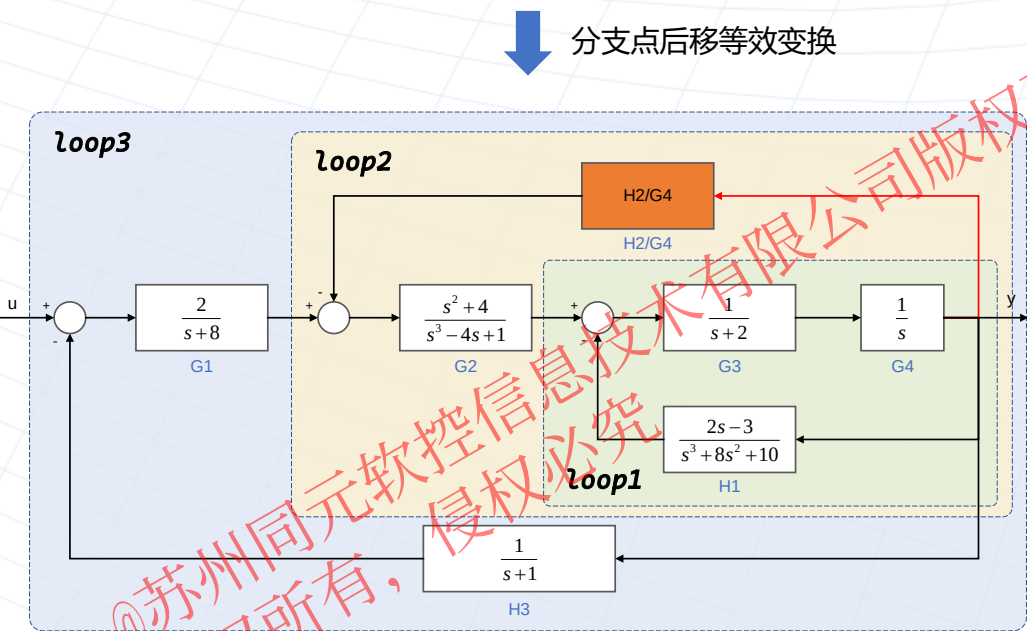
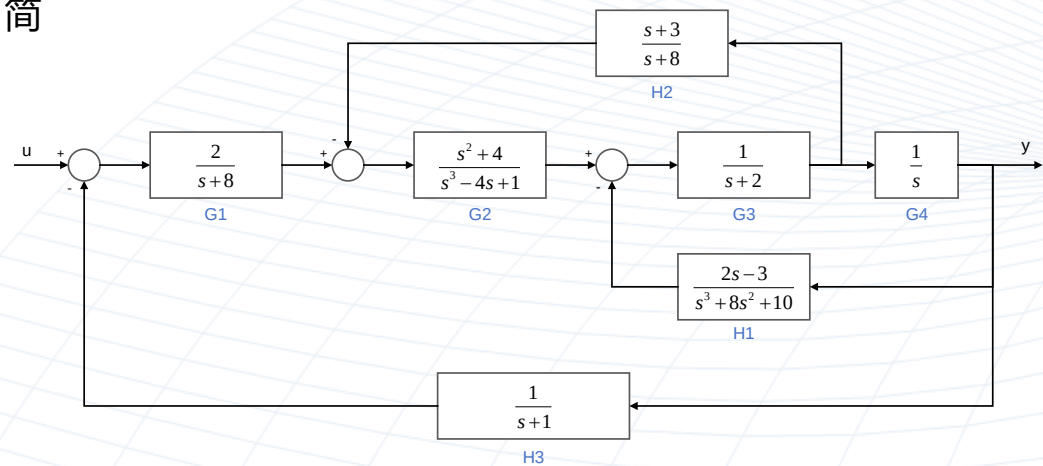
连续时间传递函数模型

```
julia> loop2
      3s^2 + 5s + 2
      -----
      3s^4 + 7s^3 + 8s^2 + 6s + 2
```

连续时间传递函数模型

2.3 线性控制系统的数学模型 – 系统连接与化简

示例2.14： 给定一个多回路控制系统的方块图，试对其进行化简



```
# 定义各模块模型
G1 = tf([2],[1,8])
G2 = tf([1,0,4],[1,0,-4,1])
G3 = tf([1],[1,2])
G4 = tf([1],[1,0])
H1 = tf([2,-3],[1,8,0,10])
H2 = tf([1,3],[1,8])
H3 = tf([1],[1,1])

# 有内到外计算各回路模型
loop1 = feedback(G3*G4,H1)
loop2 = feedback(G2*loop1,H2/G4)
loop3 = feedback(G1*loop2,H3)
```

↓ 输出

```
julia> loop1
      s^3 + 8s^2 + 10
-----
s^5 + 10s^4 + 16s^3 + 10s^2 + 22s - 3

连续时间传递函数模型

julia> loop2
      s^6 + 16s^5 + 68s^4 + 74s^3 + 336s^2 + 40s + 320
-----
s^9 + 18s^8 + 93s^7 + 78s^6 - 236s^5 - 229s^4 - 168s^3 - 550s^2 + 389s - 24

连续时间传递函数模型

julia> loop3
      2s^7 + 34s^6 + 168s^5 + 284s^4 + 820s^3 + 752s^2 + 720s + 640
-----
s^11 + 27s^10 + 263s^9 + 1059s^8 + 1210s^7 - 1727s^6 - 4085s^5 - 3758s^4 - 5757s^3 - 251s^2 + 2976s + 448

连续时间传递函数模型
```



2.3 线性控制系统的数学模型 – 系统连接与化简

示例2.15: 考虑以下系统模 $G(s) = \frac{5s^3 + 50s^2 + 155s + 150}{s^4 + 11s^3 + 41s^2 + 61s + 30}$

```
Gr = tf([5,50,155,150],[1,11,41,61,30])
GTrans = zpk(G)
```

如果不对其进行任何变换，无法发现该模型有哪些特点

$$5.0 \frac{(1.0s + 5.0)(1.0s + 3.000000000000009)(1.0s + 1.999999999999996)}{(1.0s + 4.999999999999996)(1.0s + 3.000000000000004)(1.0s + 1.999999999999982)(1.0s + 1.000000000000009)}$$

连续时间传递函数模型

从零极点模型发现，系统在 $s = -2$ 、 -3 、 -5 处有近似相同的零极点。在数学上就可以直接进行对消

```
# 系统的最小实现
Gr = minreal(G)
```

$$\frac{5.0}{1.0s + 1.0000000000000009}$$

连续时间传递函数模型

```
A = [-6 -1.5 2 4 9.5; -6 -2.5 2 5 12.5;
      -5 0.25 -0.5 3.5 9.75; -1 0.5 0 -1 1.5;
      -2 -1 1 2 3]
B = [6 4; 5 5; 3 4; 0 2; 3 1]
C = [2 0.75 -0.5 -1.5 -2.75; 0 -1.25
      1.5 1.5 2.25]
D = 0
G = ss(A,B,C,D)
Gr = minreal(G)
```

示例2.16: 考虑以下 5 阶双输入双输出系统的最小实现

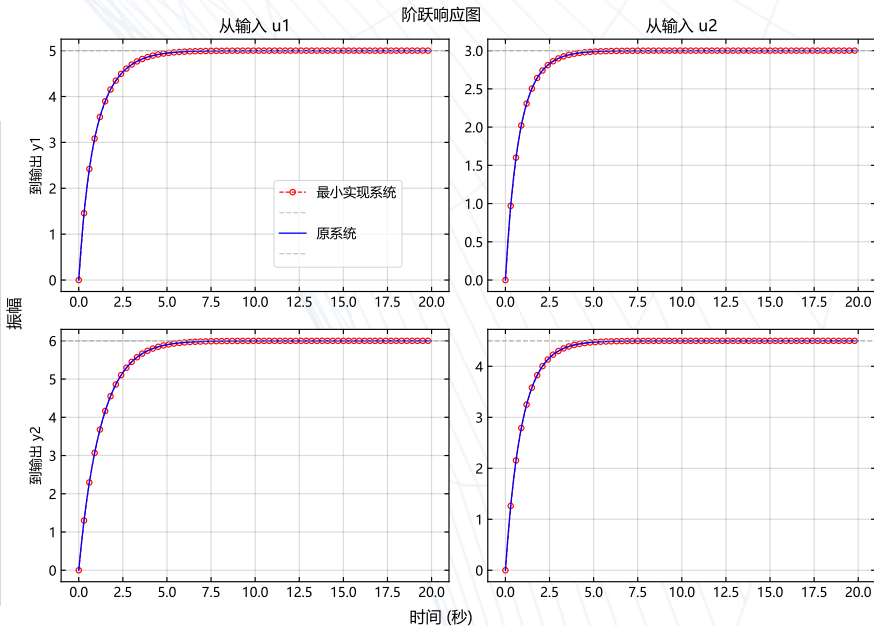
$$\dot{x} = \begin{bmatrix} -6 & -1.5 & 2 & 4 & 9.5 \\ -6 & -2.5 & 2 & 5 & 12.5 \\ -5 & 0.25 & -0.5 & 3.5 & 9.75 \\ -1 & 0.5 & 0 & -1 & 1.5 \\ -2 & -1 & 1 & 2 & 3 \end{bmatrix} x + \begin{bmatrix} 6 & 4 \\ 5 & 5 \\ 3 & 4 \\ 0 & 2 \\ 3 & 1 \end{bmatrix} u$$
$$y = \begin{bmatrix} 2 & 0.75 & -0.5 & -1.5 & -2.75 \\ 0 & -1.25 & 1.5 & 1.5 & 2.25 \end{bmatrix} x$$

```
A =
-1.2278481012658258 -0.191306397507518 2.7601986869279136
0.37367324372964017 -1.6275134947441525 0.8611642129490422
0.19052992136657032 -0.21798219140272376 -1.1446384039900197
-8.888194417315589 -7.200562565926553
0.0 -3.186204440110084
0.0 0.0
C =
-0.675052740555615 0.27015199741581764 0.4344110957473698
-0.5625439504630119 -0.29796176185568113 -0.08835479913505939
D =
0.0 0.0
0.0 0.0
```

连续时间状态空间模型

5 阶系统消除了 2 个状态变量，得到的最小实现模型为 3 阶

原系统与最小实现系统的阶跃响应对比



2.3 线性控制系统的数学模型 – 系统连接与化简

在模型化简中，均衡实现是状态方程的一种非常实用的表示形式，该模型可以将各个状态变量在整个控制系统中的重要程度明确表示出来。Syslab控制工具箱提供 `balreal()` 函数可以将已知模型进行均衡实现

示例2.17：考虑将以下模型进行简化处理

$$G(s) = \frac{s^3 + 11s^2 + 36s + 26}{s^4 + 14.6s^3 + 74.96s^2 + 153.7s + 99.65}$$

①. 将上述模型先进行一个均衡状态空间实现

```
G = tf([1, 11, 36, 26], [1, 14.6, 74.96, 153.7, 99.65])
# 均衡状态空间模型实现
Gb, g = balreal(ss(G))

julia> Gb
A =
-3.6014357728975983  0.8212109733891815 -0.61633956820488  0.05831495258992703
-0.8212109733891765 -0.592971132963471  1.0273084167890398 -0.09033395280227732
-0.6163395682048569 -1.027308416789046 -5.9138149392968336  1.1271647024056717
-0.05831495258987857 -0.09033395280224091 -1.1271647024054359 -4.491778154842064
B =
-1.0019840366001795
-0.1064122797530519
-0.0861241166324691
-0.00811170666811537
C =
-1.0019840366001798  0.10641227975305245 -0.08612411663247543  0.008111706668125246
D =
0.0

连续时间状态空间模型

julia> g
4-element Vector{Float64}:
 0.13938496656762894
 0.009548165713944949
 0.0006271217092401019
 7.324469597705166e-6
```

通过 Gramian 矩阵的对角线向量 g，判定均衡实现的后 2 个状态对系统影响较小

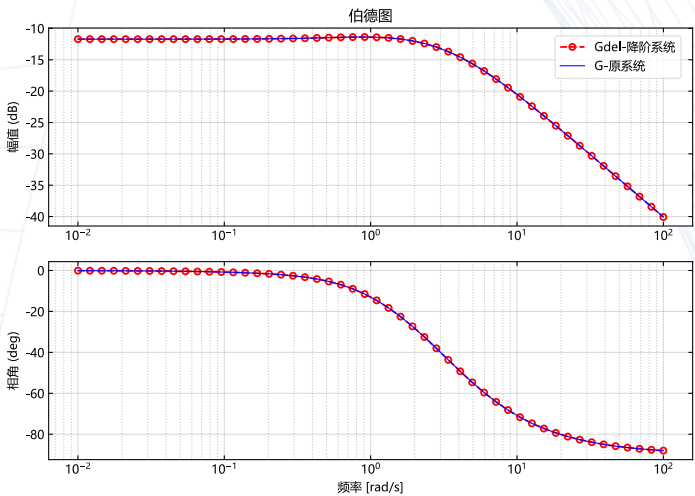
②. 通过 `modred()` 函数从均衡实现中消除对系统影响较小的状态

```
# 消除影响较小的两个状态
Gdel = modred(Gb, [3,4], method=:Truncate)

julia> Gdel
A =
-3.6014357728975983  0.8212109733891815
-0.8212109733891765 -0.592971132963471
B =
-1.0019840366001795
-0.1064122797530519
C =
-1.0019840366001798  0.10641227975305245
D =
0.0

连续时间状态空间模型
```

消除2个影响小的状态变量后得到的系统模型



原始 4 阶系统与简化的 2 阶系统在频率上的响应几乎一致

2.3 线性控制系统的数学模型 – 系统连接与化简

系统连接与化简相关主要Syslab函数，其余函数请参阅Syslab-控制系统工具箱帮助文档

函数及调用方式	说明
<code>sys = feedback(sys1, sys2)</code>	回模型对象 <code>sys1</code> 、 <code>sys2</code> 的负反馈互连的模型对象 <code>sys</code>
<code>sys = parallel(sys1, sys2)</code>	两个模型的并联连接
<code>sys = append(sys1, sys2, ... , sysN)</code>	通过增加模型的输入和输出对模型进行分组
<code>sysc = connect(blksys, connections, inputs, outputs)</code>	动态系统的相互连接
<code>sys = lft(sys1, sys2, nu, ny)</code>	两个模型的广义反馈互连(Redheffer星积)
<code>sys = series(sys1, sys2)</code>	两个模型的串联连接
<code>sysb, g = balreal(sys)</code> <code>sysb, g, T = balreal(sys)</code>	基于gramian的状态空间实现的输入/输出平衡
<code>sysr = minreal(sys)</code>	最小实现或极点、零点抵消
<code>rsys = modred(sys, elim, method = value)</code>	从状态空间模型中消除状态
<code>msys = sminreal(sys)</code>	结构极点/零点对消

03

线性控制系统分析

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司
版权所有，侵权必究

3.1 线性控制系统分析 – 时域分析

时域分析是一种最直观、最直接的分析。一般可以为控制系统预先规定一些特殊的试验输入信号，然后比较各种系统对这些信号的响应情况。

经常采用的试验输入信号

信号名称	时域表达式	复频域表达式
单位阶跃	$1(t), t \geq 0$	$\frac{1}{s}$
单位斜坡	$t, t \geq 0$	$\frac{1}{s^2}$
单位加速度	$\frac{1}{2}t^2, t \geq 0$	$\frac{1}{s^3}$
单位脉冲	$\delta(t), t = 0$	1
正弦函数	$A \sin \omega t$	$\frac{A\omega}{s^2 + \omega^2}$

一. 阶跃响应 – step()

- 对于稳定系统，通常在系统阶跃响应曲线上来定义系统动态性能指标
- 系统的单位阶跃响应不仅完整反映了系统的动态特性，而且反映了系统在单位阶跃信号输入下的稳定状态。同时，单位阶跃信号又是一个最简单、最容易实现的信号。

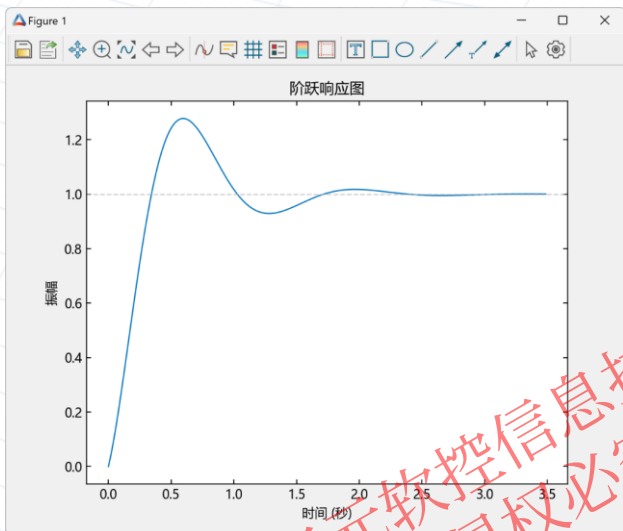
step调用方式	说明
step(sys)	计算并直接返回系统阶跃响应图。其中sys可以是：tf、ss、zpk
step (sys, t)	计算向量 t 指定时间内的阶跃响应。t为标量：计算[0,t]内的响应，t为向量，计算各点的阶跃响应。示例： - step(G, 5) - step(G, 0:0.1:10)
step(____, fmt)	计算并直接返回系统阶跃响应图。fmt为绘图样条属性设置字符串。示例： - step(G, "-bo", linewidth = 1, markersize = 5, ...) - step(G, "-r", linewidth = 1, ishold = true, ...) # 将图形绘制在已有figure上
res = step(sys, t, fig = false)	计算阶跃响应数据，生成 res 响应数据为SimResult结构体，不出图。其中： - res.t:时间向量 - res.y:响应数组
y,t,x = step(sys, t, fig = false)	计算阶跃响应数据，y 为响应数据数组，t 为时间向量，x 为状态数组

3.1 线性控制系统分析 – 时域分析

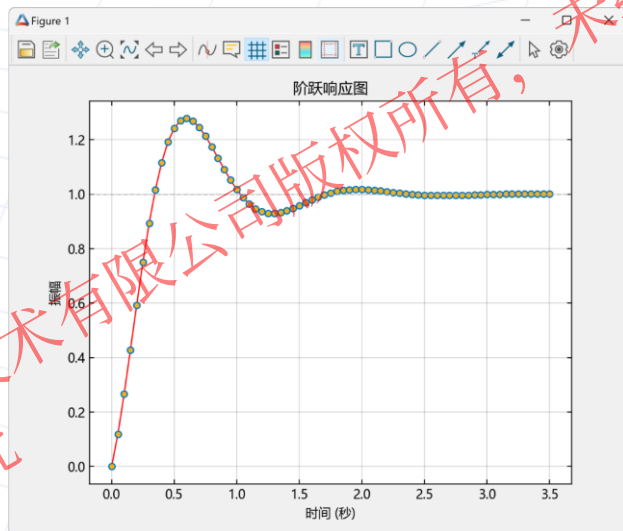
一. 阶跃响应 – step()

示例3.1: 计算并绘制以下系统的阶跃响应 $G(s) = \frac{2s + 25}{s^2 + 4s + 25}$

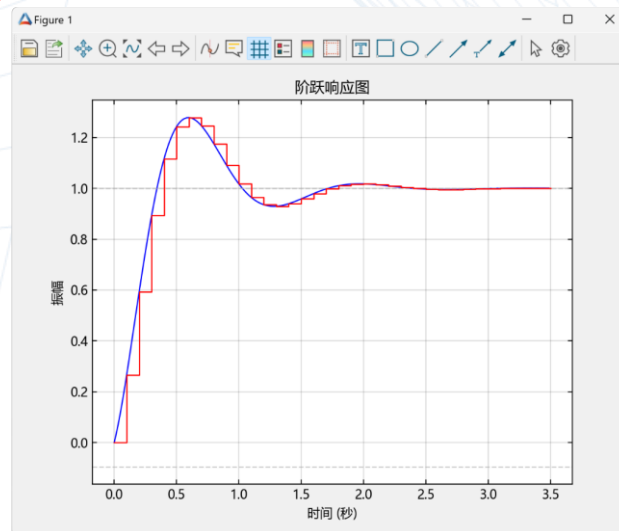
```
G = tf([2 25],[1 4 25])  
# 阶跃响应基础绘图  
step(G, 3.5)
```



```
# 图形修饰  
step(G, 0:0.05:3.5, "-ro", linewidth = 1,  
      markersize = 5, markeredgecolor =  
      "#0072BD", markerfacecolor = "#EDB120")  
grid("on")
```



```
# 多个系统响应绘图叠加  
step(G, 3.5, "-b")  
step(c2d(G, 0.1), 3.5, "-r", ishold =  
      true)  
grid("on")
```



```
# 直接获取阶跃响应数据  
y, t = step(G, fig = false)
```

```
julia> t  
101-element Vector{Float64}:  
 0.0  
 0.035  
 0.07  
 0.10500000000000001  
 ⋮  
 3.43  
 3.4650000000000003  
 3.5000000000000004  
julia> y  
1×101 Matrix{Float64}:  
 0.0 0.0795734 0.174988  
 0.281454 0.394494 0.510026  
 0.624432 0.734596 0.837934  
 0.932392 1.01644 1.08903  
 1.14961 1.198 ... 0.999587  
 0.999952 1.00027 1.00054  
 1.00076 1.00093 1.00105  
 1.00112 1.00115 1.00115  
 1.00111 1.00105 1.00096  
 1.00086
```

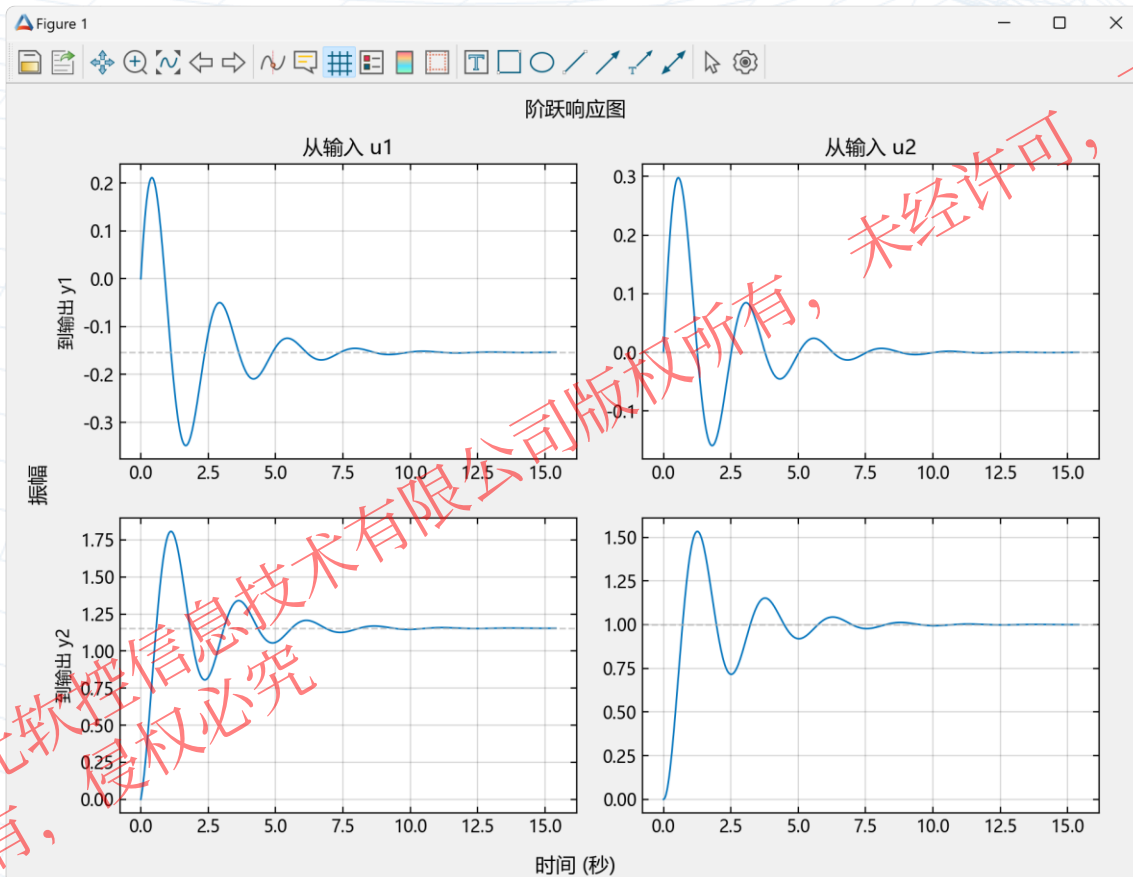
3.1 线性控制系统分析 – 时域分析

一. 阶跃响应 – step()

示例3.2: 计算并绘制双输入双输出系统的阶跃响应

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} -1 & -1 \\ 6.5 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$
$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

```
# 定义系统矩阵
A = [-1 -1; 6.5 0]
B = [1 1; 1 0]
C = [1 0; 0 1]
D = zeros(2,2)
# 创建状态空间模型
G = ss(A,B,C,D)
# 计算系统阶跃响应
step(G,15)
grid("on")
```



```
# 获取输出数据
y, t = step(G, fig = false)
```

```
julia> t
0.0 0.030805850470027103:15.402925235013551

julia> y
2×1301×2 Array{Float64, 3}:
[:, :, 1] =
 0.0  0.0830874 ... -0.153846 -0.153846 -0.153846
 0.0  0.117899   ...  1.15385  1.15385  1.15385

[:, :, 2] =
 0.0  0.0871829 ... -9.71445e-17 -9.71445e-17
 0.0  0.0266207 ...  1.0        1.0
```

对于MIMO系统的响应数据 y 为一个**三维数组**，其维度为：

$N_y * N * N_u$ ，其中：

- N_y 为系统输出数量
- N 为时间向量长度
- N_u 为系统输入数量

因此，通过 $y[i, :, j]$ 可取出第 j 个输入到第 i 个输出的阶跃响应数据向量。

3.1 线性控制系统分析 – 时域分析

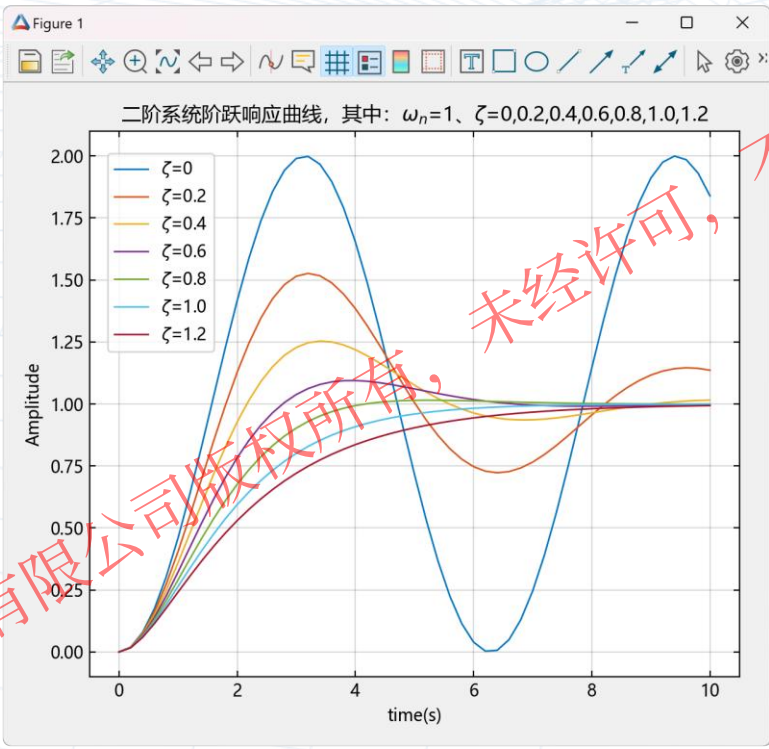
一. 阶跃响应 – step()

示例3.3：计算并绘制标准二阶系统阶跃响应曲线及响应面 $G(s)=\frac{1}{s^2+2\zeta s+1}$

① 阶跃响应曲线计算与绘制

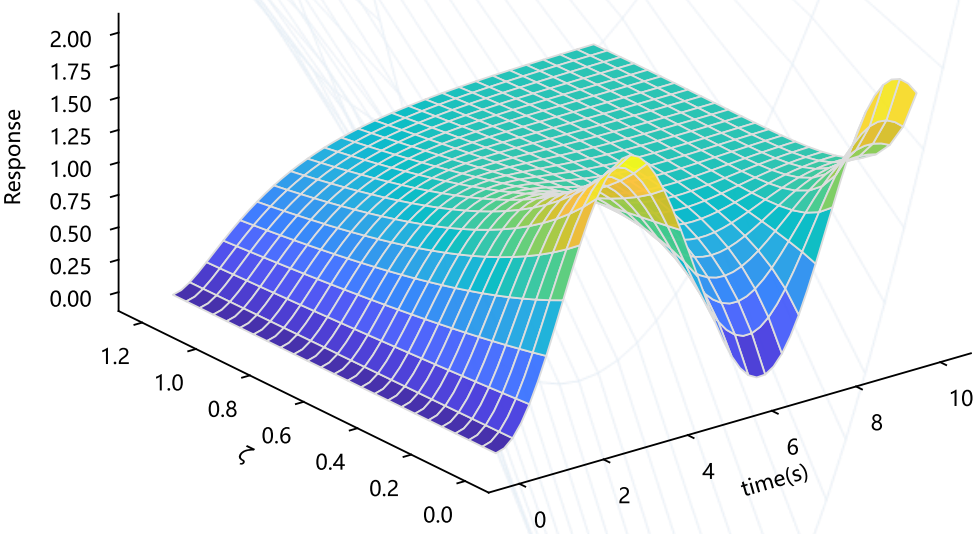
```
# 时间向量定义
t = 0:0.2:10
# 阻尼比定义
zeta = 0:0.05:1.2 # \zeta<Tab>
# 传递函数及响应变量预定义
num = [0];
den = [0 0 0];
y = zeros(length(t), length(zeta))
ty = zeros(length(t), length(zeta))
# 计算不同阻尼比下的二阶系统阶跃响应
for i in 1:length(zeta)
    num = [1]
    den = [1, 2 * zeta[i], 1]
    y[:, i], ty[:, i] = step(tf(num, den), t, fig=false)
end

# 绘制\zeta = 0的阶跃响应曲线
plot(t, y[:, 1])
hold("on")
# 绘制\zeta = 0.2, 0.4, 0.6, 0.8, 1.0, 1.2的阶跃响应曲线
for j in 5:4:length(zeta)
    plot(t, y[:, j])
end
# 图形修饰
grid("on")
title(raw"二阶系统阶跃响应曲线, 其中: $\omega_n=1, \zeta=0, 0.2, 0.4, 0.6, 0.8, 1.0, 1.2$")
xlabel("time(s)")
ylabel("Amplitude")
legend(raw"\zeta=0", raw"\zeta=0.2", raw"\zeta=0.4", raw"\zeta=0.6", raw"\zeta=0.8", raw"\zeta=1.0", raw"\zeta=1.2")
```



② 阶跃响应面绘制

```
# 构造\zeta, t构成的网格
zeta, T = meshgrid2(zeta, t)
# 三维响应面绘制
s = mesh(T, zeta, y; facealpha=0.95)
grid("off")
# 三维响应面图形修饰
xlabel("time(s)")
ylabel(raw"\zeta")
zlabel("Response")
s.set_facecolor("flat")
s.set_edgecolor("#ddddd")
plt_update()
```

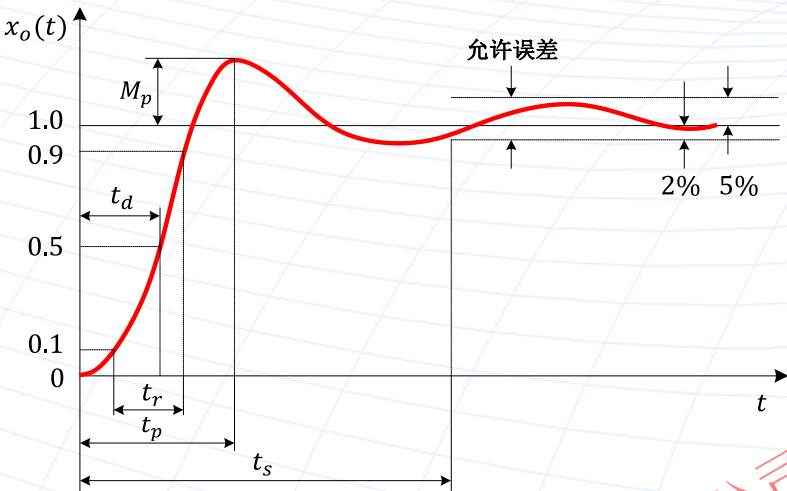


3.1 线性控制系统分析 – 时域分析

一. 阶跃响应 – step()

时域分析性能指标

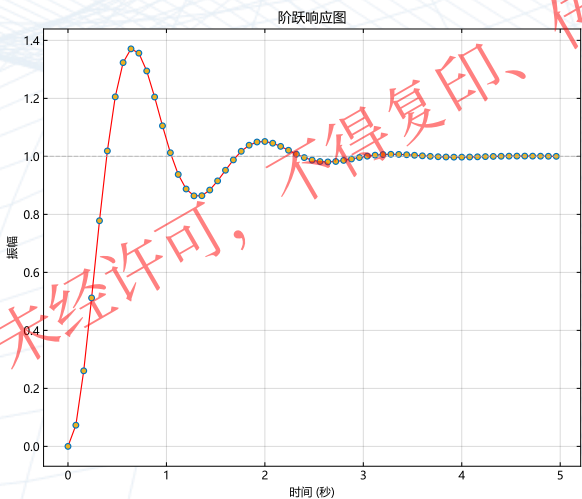
通常控制系统的性能指标以系统对单位阶跃输入量的瞬态响应形式给出



时域性能指标	含义
上升时间 t_r	响应曲线由稳态值的10%上升到稳态值得90%所需的时间
峰值时间 t_p	响应曲线从零上升到第一个峰值所需要的时间
最大超调量 M_p	响应曲线最大峰值与稳态值之差
调整时间 t_s	响应曲线达到并一直保持在允许误差范围内的最短时间
延迟时间 t_d	响应曲线从0上升到稳态值的50%所需要的时间

通过 `stepinfo()` 函数计算并获取动态系统阶跃响应特性

示例3.4：获取系统的阶跃响应特性 $G(s) = \frac{25}{s^2 + 3s + 15}$



```
G = tf([25],[1 3 15])
# 计算并获取系统阶跃响应特性
res = stepinfo(G)
# 上升时间
res.RiseTime
# 最大超调
res.Overshoot
# 峰值
res.Peak
# 峰值时间
res.PeakTime
# 调整时间
res.SettlingTime
```

更多设置选项，参考帮助文档

```
julia> # 上升时间
julia> res.RiseTime
1x1 Matrix{Float64}:
 0.26472503148285226

julia> # 最大超调
julia> res.Overshoot
1x1 Matrix{Float64}:
 37.1410271661408

julia> # 峰值
julia> res.Peak
1x1 Matrix{Float64}:
 1.371410271661408

julia> # 峰值时间
julia> res.PeakTime
1x1 Matrix{Float64}:
 0.6447238260382373

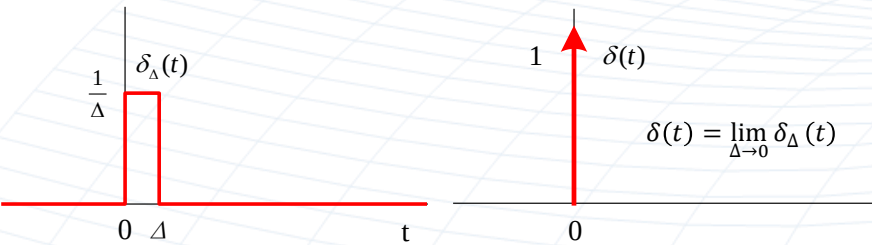
julia> # 调整时间
julia> res.SettlingTime
1x1 Matrix{Float64}:
 2.246034069329275
```



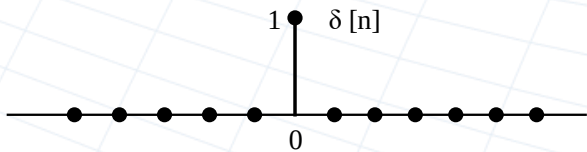
3.1 线性控制系统分析 – 时域分析

二. 脉冲响应 – impulse()

系统的脉冲响应（或称为冲激响应）可以用 `impulse()` 函数进行计算并绘制脉冲响应图。



连续时间单位脉冲信号是一个持续时间为 Δ 的短脉冲，对于任意的 Δ 值，其面积均为1，随着 $\Delta \rightarrow 0$ ， $\delta_{\Delta}(t)$ 变得越来越窄，越来越高，但单位面积不变



离散时间单位脉冲信号可以表达为：

$$u[n] = \begin{cases} 0, & n \neq 0 \\ 1, & n = 0 \end{cases}$$

impulse 调用方式	说明
<code>impulse(sys)</code>	计算并直接返回系统脉冲响应图。其中sys可以是：tf、ss、zpk
<code>impulse (sys,t)</code>	计算向量 t 指定时间内的脉冲响应。t为标量：计算[0,t]内的响应，t为向量，计算各点的脉冲响应。示例： <code>impulse(G,5)</code> <code>impulse(G,0:0.1:10)</code>
<code>impulse(____,fmt)</code>	计算并直接返回系统脉冲响应图。fmt为绘图样条属性设置字符串。示例： <code>impulse(G,"-bo",linewidth = 1,markersize = 5,...)</code> <code>impulse(G,"-r",linewidth = 1, ishold = true,...) #</code> 将图形绘制在已有figure上
<code>res = impulse (sys, t, fig = false)</code>	计算脉冲响应数据，生成 res 响应数据为SimResult结构体，不出图。其中： <code>res.t</code>:时间向量 <code>res.y</code>:响应数组
<code>y,t,x = impulse (sys, t, fig = false)</code>	计算阶跃响应数据，y 为响应数据数组，t 为时间向量，x 为状态数组

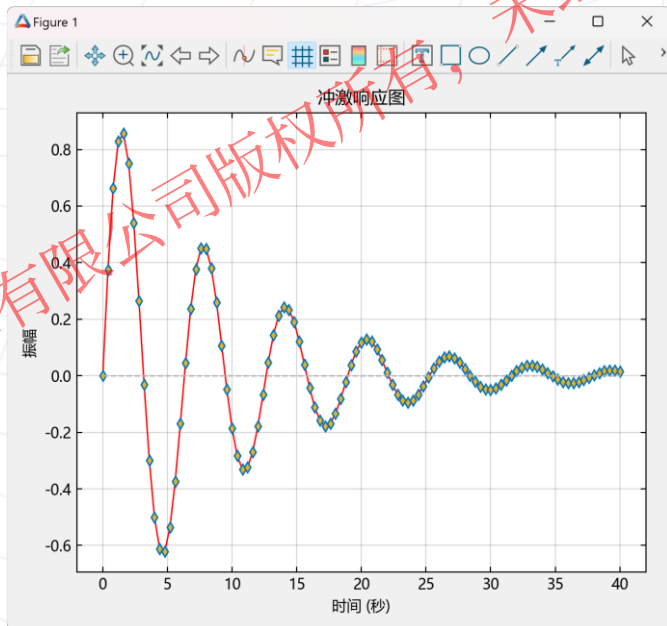
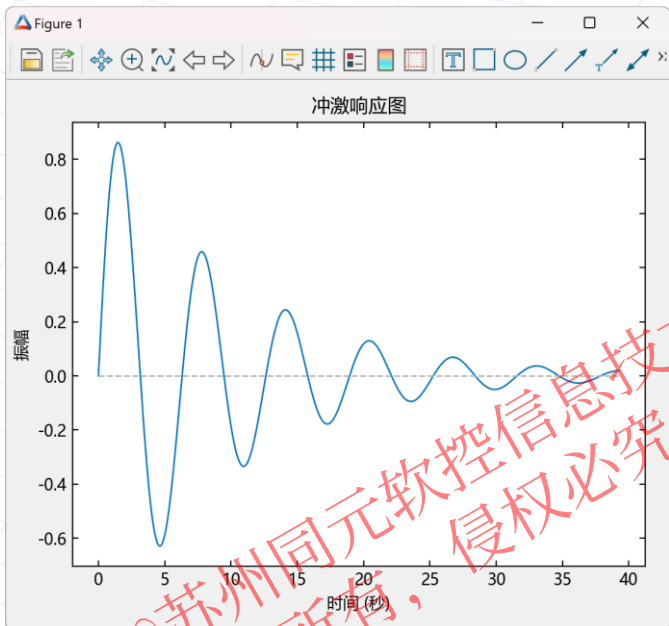
3.1 线性控制系统分析 – 时域分析

二. 脉冲响应 – impulse()

示例3.5: 计算以下系统的脉冲响应 $H(s) = \frac{1}{s^2 + 0.2s + 1}$

```
H = tf([1],[1,0.2,1])  
impz(H)
```

```
# 图形修饰  
H = tf([1],[1,0.2,1])  
impz(H,0:0.4:40,"-rd",linewidth = 1,  
      markersize = 5, markeredgecolor =  
      "#0072BD", markerfacecolor = "#EDB120")  
grid("on")
```



求脉冲响应的另一种方法

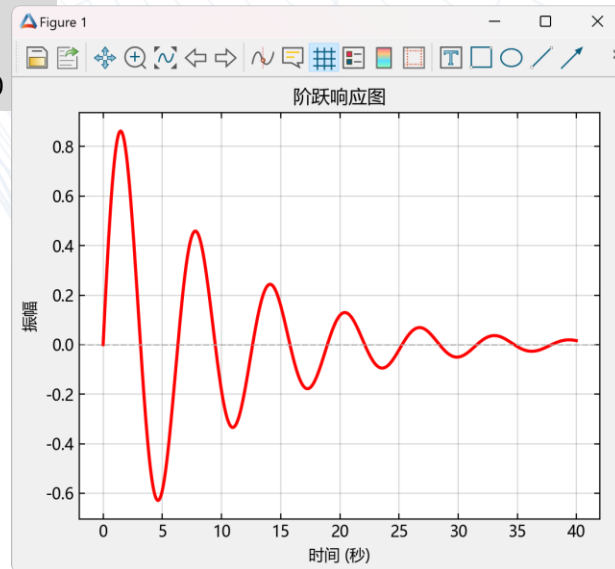
考虑系统的脉冲响应表达式为: $Y(s) = G(s) \cdot U(s)$, 其中脉冲信号 $U(s) = 1$

$$Y(s) = G(s) \cdot 1 = s \cdot G(s) \cdot \frac{1}{s}$$

因此, 求取 $G(s)$ 的脉冲响应, 可以转化为求取 $s \cdot G(s)$ 的单位阶跃响应

示例3.6: 针对示例3.5, 通过阶跃函数求取其脉冲响应

```
s = tf('s')  
H = tf([1],[1,0.2,1])  
# 通过阶跃函数step求取 H 的脉冲响应  
step(s*H,40,"-r",linewidth = 1)
```



两种计算方式等价

3.1 线性控制系统分析 – 时域分析

三. 斜坡信号响应

Syslab控制工具箱没有提供斜坡信号响应函数，同样可以考虑使用上面的方法进行求取。考虑系统的斜坡响应表达式为： $Y(s) = G(s) \cdot U(s)$ ，其中斜坡信号： $U(s) = \frac{1}{s^2}$

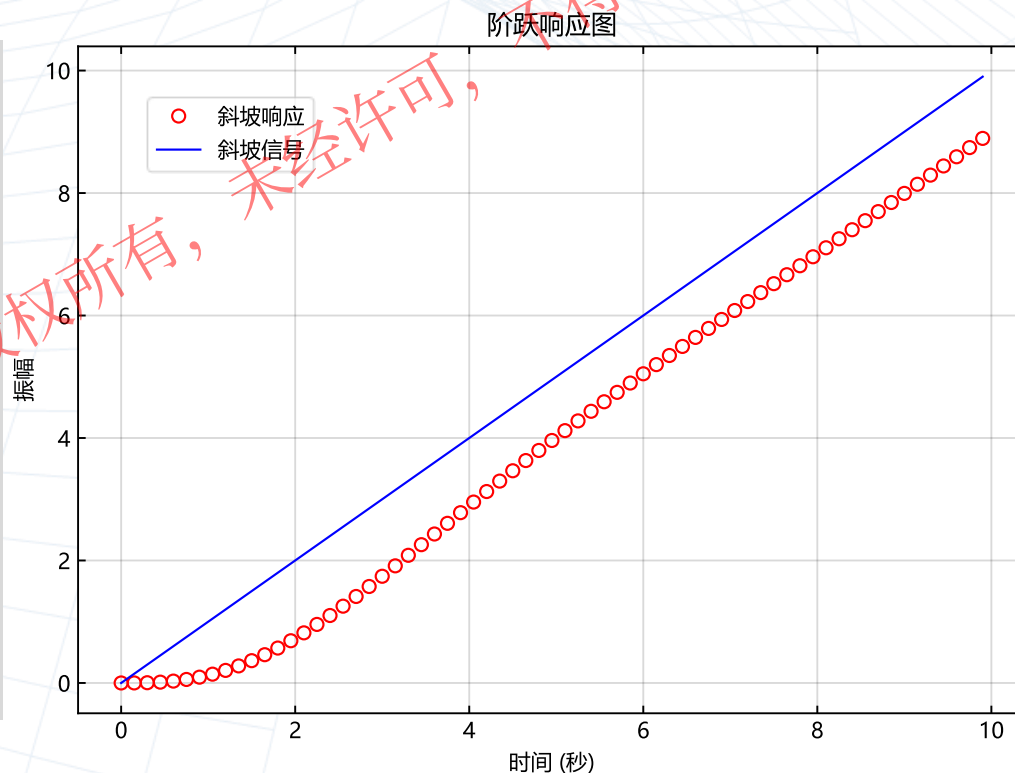
$$Y(s) = G(s) \cdot \frac{1}{s^2} = \frac{1}{s} \cdot G(s) \cdot \frac{1}{s}$$

求取 $G(s)$ 的斜坡响应，转化为求取 $\frac{1}{s} \cdot G(s)$ 的单位阶跃响应

示例3.7：计算以下系统的斜坡响应

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -1 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u$$
$$y = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

```
# 定义Laplace算子
s = tf('s')
# 定义状态空间矩阵
A = [0 1; -1 -1]
B = [0; 1]
C = [1 0]
D = [0]
# 创建系统模型
G = ss(A,B,C,D)
# 指定时间向量
t = 0:0.15:10
# 通过step计算斜坡响应
step((tf(G)/s), t, "ro")
hold("on")
# 绘制斜坡输入信号
plot(t, t, "-b")
legend("斜坡响应", "斜坡信号")
grid("on")
```



类似的，系统对单位加速度信号的响应可以使用同样的方式计算得到

3.1 线性控制系统分析 – 时域分析

四. 对任意信号的响应 – lsim()

为了求对任意输入信号的响应，可以使用 `lsim()` 函数

lsim 调用方式	说明
<code>lsim(sys,u,t)</code>	计算并直接返回系统对输入信号 (t,u) 的时域响应图。其中 t 为时间向量, u 的维度为: $N_u \times length(t)$
<code>lsim(sys,u,t,fmt)</code>	计算并直接返回系统对输入信号 (t,u) 的时域响应图。fmt为绘图样条属性设置字符串。示例: • <code>lsim(G,u,t,"-bo",linewidth = 1,markersize = 5,...)</code> • <code>lsim(G,u,t,"-r",linewidth = 1, ishold = true,...)</code> # 将图形绘制在已有figure上
<code>lsim(sys,u,t,x0 = value)</code>	当sys是状态空间模型时，可以进一步指定初始状态值 x0 , 注意 x0 为向量形式。
<code>lsim(sys,u,t,x0 = value, fmt)</code>	当sys是状态空间模型时，可以进一步指定初始状态值 x0 , fmt为绘图样条属性设置字符串。
<code>res = lsim (sys, u, t, fig = false)</code>	计算任意信号响应数据，生成 res 响应数据为SimResult结构体，不出图。其中: • <code>res.t</code>:时间向量 • <code>res.y</code>:响应数组
<code>y,t,x = lsim (sys, u, t, fig = false)</code>	计算任意信号响应数据, y 为响应数据数组, t 为时间向量, x 为状态数组

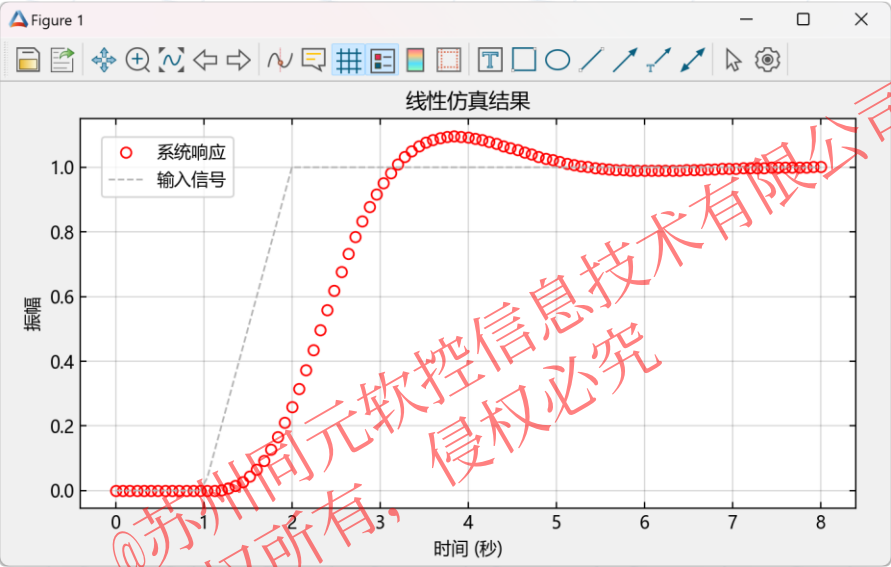


3.1 线性控制系统分析 – 时域分析

四. 对任意信号的响应 – lsim()

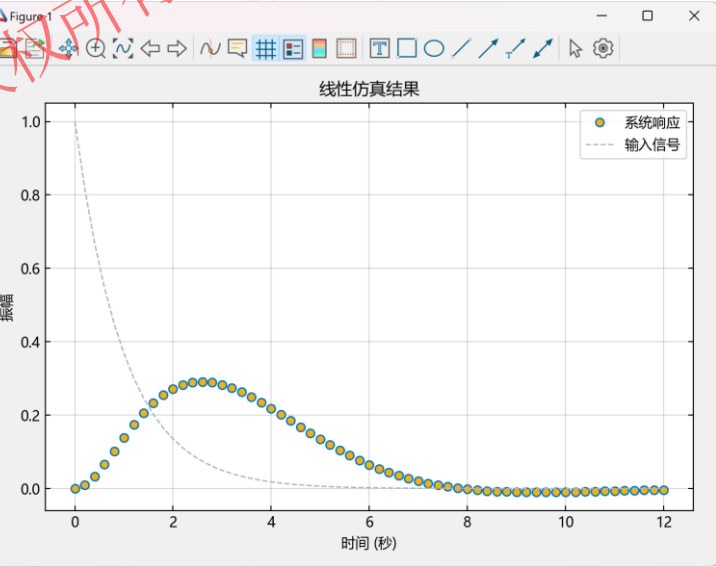
示例3.8：计算系统对自定义斜坡阶跃信号的响应，输入信号在 t = 0 时从 0 开始，在 t = 1 时从 0 开始单位斜坡 1s 到 1，然后在 1 处保持稳定。

```
# 定义系统
sys = tf([3],[1,2,3])
# 创建输入信号
t = 0:0.08:8
u = max.(0,min.(t-1,1))
# 计算系统响应
lsim(sys, reshape(u, 1, length(u)), t,"ro")
```



示例3.9：考虑以下系统在输入信号 $u = e^{-t}$ 作用下的响应情况，假设初始状态为 $x(0) = 0$

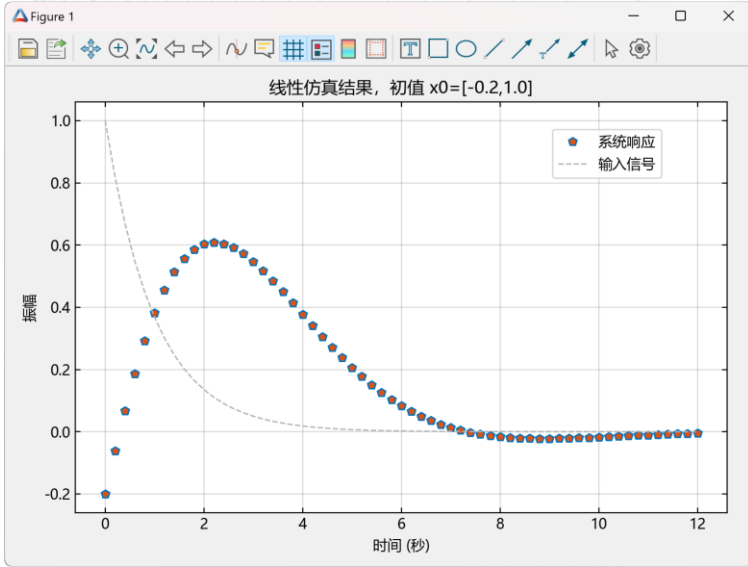
```
A = [-1 0.5;-1 0]
B = [0;1]
C = [1 0]
D = 0
G = ss(A,B,C,D)
# 创建输入信号
t = 0:0.2:12
u = exp.(-t)
u = reshape(u, 1, length(u))
lsim(G,u,t,"ro")
```



$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} -1 & 0.5 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u$$
$$y = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

改变初始状态为: $x(0) = [-0.2, 1.0]$

```
lsim(G,u,t,[-0.2,1.0],"rp",markeredgecolor = "#0072BD", markerfacecolor = "#D95319")
legend("系统响应","输入信号")
title("线性仿真结果，初值 x0=[-0.2,1.0]")
grid("on")
```



3.2 线性控制系统分析 – 频域分析

系统对正弦输入信号的**稳态响应**成为频率响应。频率特性的图形化表示常用**伯德图**、**奈奎斯特图**以及**尼克尔斯图**。

一. 伯德图 – bode()

bode 调用方式	说明
<code>bode(sys)</code>	计算系统频率响应, 并绘制系统bode图
<code>bode(sys, w)</code>	计算系统频率响应, 并绘制系统bode图, 其中 <code>w</code> 为指定的计算频率 <code>w</code> 为标量: 频率特性计算范围为$[0, w]$, Syslab将自动计算频率向量 <code>w</code> 为向量: 系统将按照 <code>w</code> 计算指定频率点的响应
<code>bode(____, fmt)</code>	计算系统频率响应, 并绘制系统bode图。fmt为绘图样条属性设置字符串。示例: <code>bode(G, w, "-bo", linewidth = 1, markersize = 5, ...)</code> <code>bode(G, w, "-r", linewidth = 1, ishold = true, ...)</code> # 将图形绘制在已有figure上
<code>mag, phase, wout = bode(sys, fig = false)</code>	计算系统频率响应数据, 不出图。其中: <code>mag</code>: 幅频特性数组, 维度为: 输出数量 * 输入数量 * 频率向量长度 <code>phase</code>: 相频特性数组, 维度为: 输出数量 * 输入数量 * 频率向量长度 <code>wout</code>: 频率向量
<code>mag, phase, wout = bode(sys, w, fig = false)</code>	计算系统频率响应数据, 不出图。其中: <code>mag</code>: 幅频特性数组, 维度为: 输出数量 * 输入数量 * 频率向量长度 <code>phase</code>: 相频特性数组, 维度为: 输出数量 * 输入数量 * 频率向量长度 <code>wout</code>: 频率向量

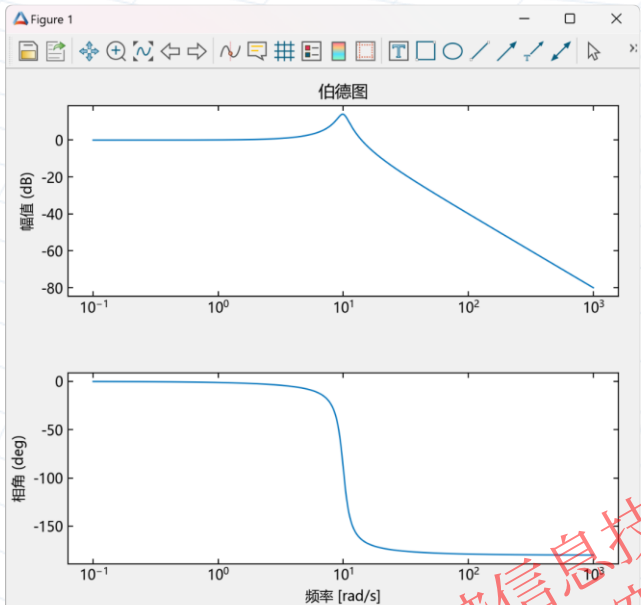
3.2 线性控制系统分析 – 频域分析

一. 伯德图 – bode()

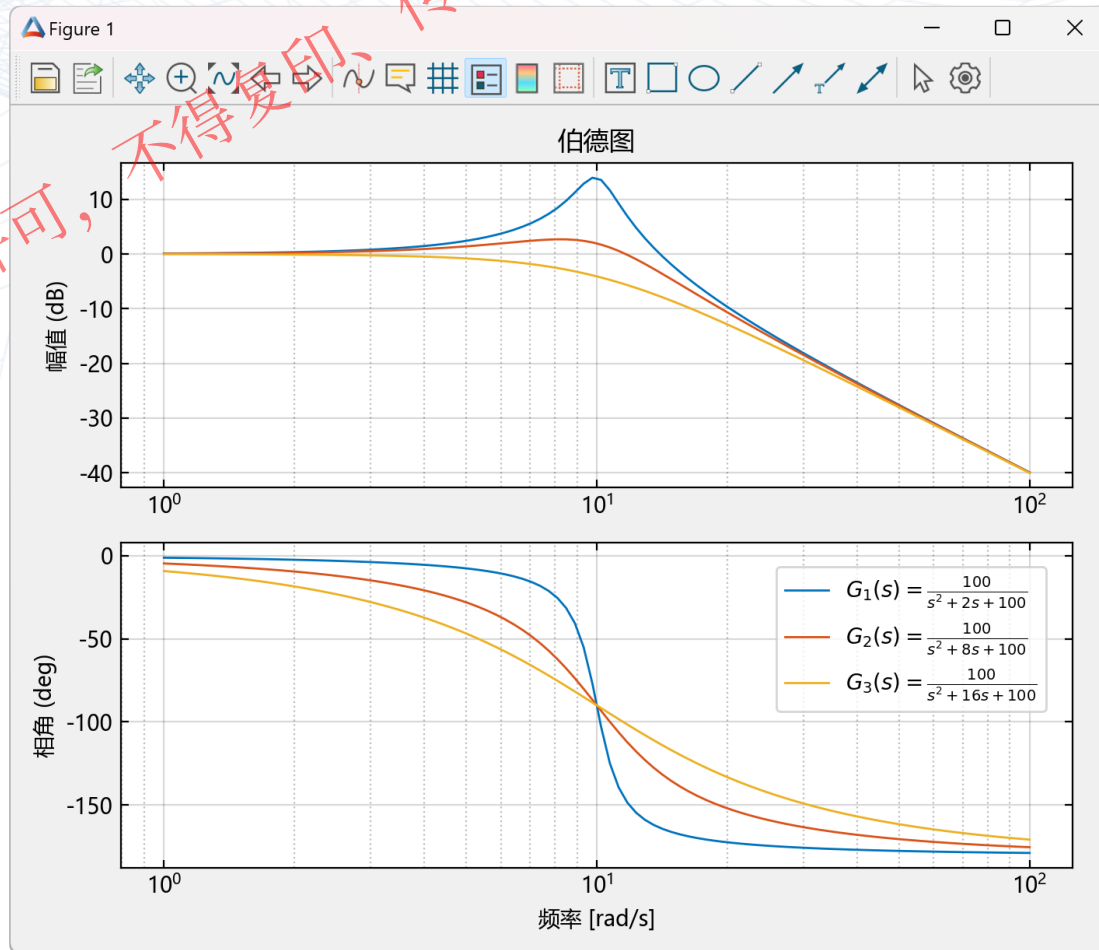
示例3.10: 计算系统频率响应并绘制bode图

$$G(s) = \frac{100}{s^2 + 4s + 100}$$

```
G = tf([100],[1,2,100])  
bode(G)
```



```
# 定义频率向量  
w = logspace(0,2,100)  
# 定义不同阻尼比的系统模型  
G1 = tf([100],[1,2*0.1*10,100])  
G2 = tf([100],[1,2*0.4*10,100])  
G3 = tf([100],[1,2*0.8*10,100])  
# bode图绘制  
bode(G1,w,linewidth = 1)  
bode(G2,w,linewidth = 1, ishold = true)  
bode(G3,w,linewidth = 1, ishold = true)  
# bode图网格开启  
bodegrid(true)  
# 定义legend  
G1l = raw"$G_1\left(s\right) = \frac{100}{s^2 + 2s + 100}$"  
G2l = raw"$G_2\left(s\right) = \frac{100}{s^2 + 8s + 100}$"  
G3l = raw"$G_3\left(s\right) = \frac{100}{s^2 + 16s + 100}$"  
legend(G1l,G2l,G3l)
```



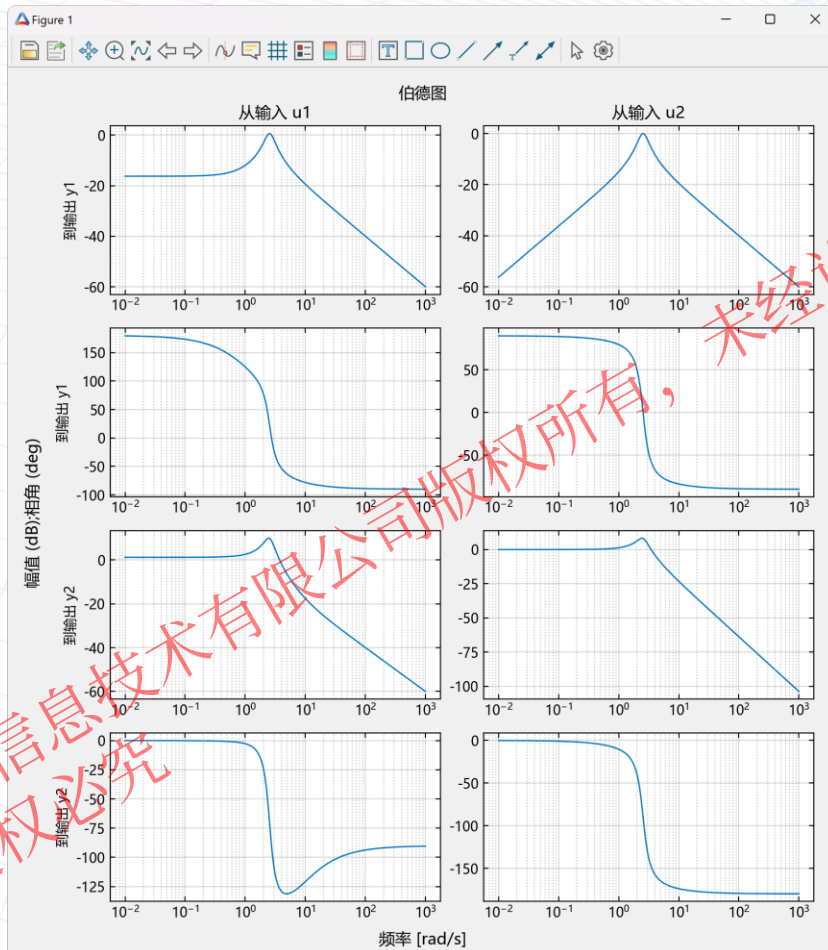
3.2 线性控制系统分析 – 频域分析

一. 伯德图 – bode()

示例3.11: 计算下面MIMO系统频率响应并绘制bode图

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} -1 & -1 \\ 6.5 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$
$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

```
# 定义系统矩阵
A = [-1 -1; 6.5 0]
B = [1 1; 1 0]
C = [1 0; 0 1]
D = zeros(2,2)
# 创建状态空间模型
G = ss(A,B,C,D)
# 绘制系统bode图
bode(G)
bodegrid(true)
```



计算并获取系统频率响应数据

```
w = logspace(-2,3,100)
mag, phase, wout = bode(G,w,fig = false)
```

```
julia> mag
2×2×100 Array{Float64, 3}:
[:, :, 1] =
 0.153856  0.00153848
 1.15386   1.00001
[:, :, 2] =
 0.153859  0.00172822
 1.15387   1.00002
[:, :, 3] =
 0.153862  0.00194136
 1.15387   1.00002
... ..
```

```
julia> phase
2×2×100 Array{Float64, 3}:
[:, :, 1] =
 179.339    89.9119
 -0.0117543 -0.0881486
[:, :, 2] =
 179.257    89.901
 -0.0132043 -0.0990199
[:, :, 3] =
 179.166    89.8888
 -0.0148333 -0.111232
... ..
```

系统频率响应数据mag、phase均为**三维数组**

其维度: $N_y * N_u * \text{length}(wout)$, 其中:

- N_y 为系统输出数量
- N_u 为系统输入数量
- $\text{length}(wout)$ 为频率向量长度

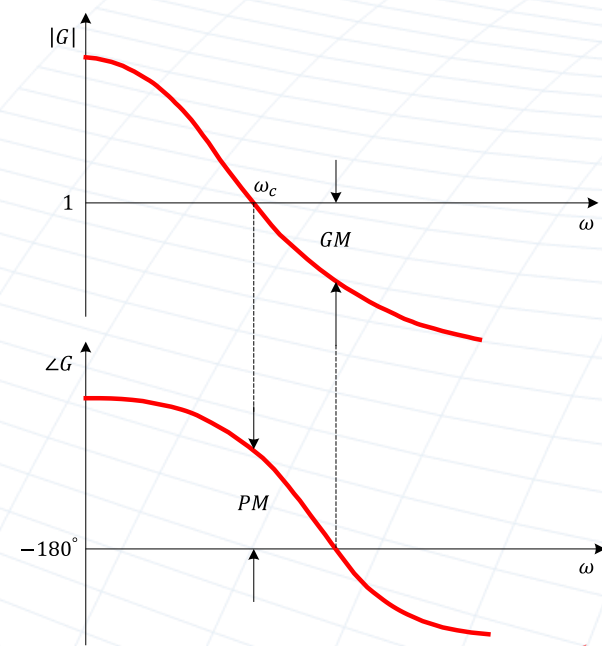
因此, 通过 `mag[i,j,:]` 可取出第 **j** 个输入到第 **i** 个输出的幅频响应数据向量

3.2 线性控制系统分析 – 频域分析

一. 伯德图 – bode()

系统频域指标

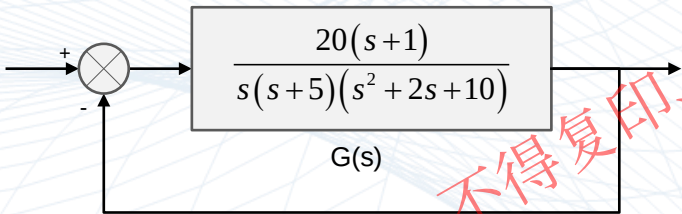
①. 开环频域指标



开环频域指标	含义
开环剪切频率 ω_c	开环频率特性幅值为1对应的频率
幅值裕度 GM	在相角等于 -180° 的频率上，幅频特性 $ G(j\omega) $ 的倒数
相位裕度 PM	剪切频率处，相频特性距 -180° 线的相位差

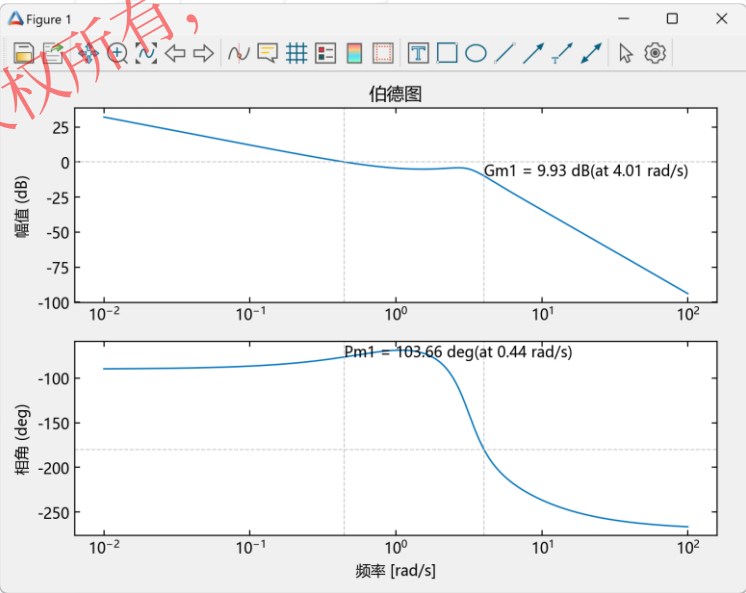
通过 `margin()` 函数计算系统开环频率指标

示例3.12：设一闭环系统如下，绘制其开环传递函数bode图，并确定其增益裕度、相位裕度



```
s = tf('s')
Gopenloop = 20*(s+1)/(s*(s+5)*(s^2+2*s+10))
margin(Gopenloop)
```

```
Gm, Pm, Wcg, Wcp = margin(Gopenloop, fig = false)
```



```
julia> Gm #绝对单位，可通过 20*log10.(Gm)计算dB值
1x1 Matrix{Float64}:
 3.17675128297679

julia> Pm # 相位裕度
1x1 Matrix{Float64}:
103.66528865343292

julia> Wcg # 增益裕度对应频率
1x1 Matrix{Float64}:
 4.02997029195757

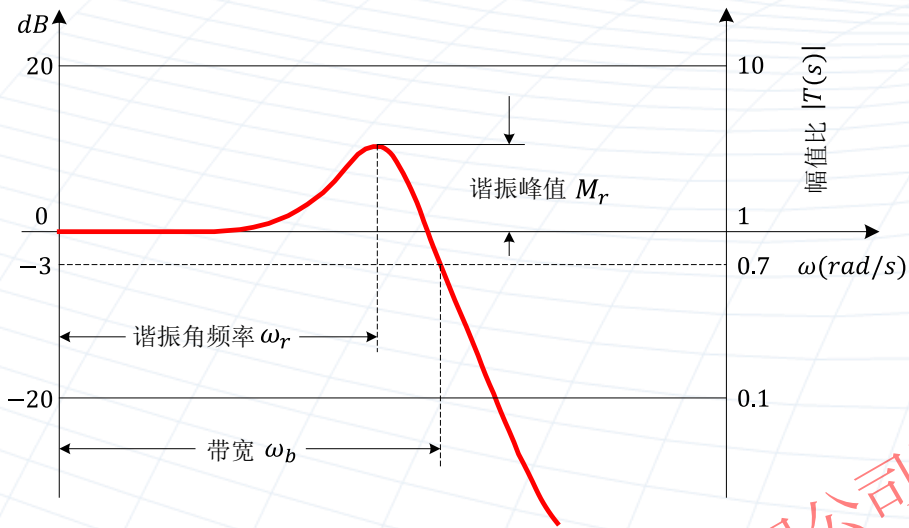
julia> Wcp # 开环剪切频率
1x1 Matrix{Float64}:
 0.4429643273018798
```

3.2 线性控制系统分析 – 频域分析

一. 伯德图 – bode()

系统频域指标

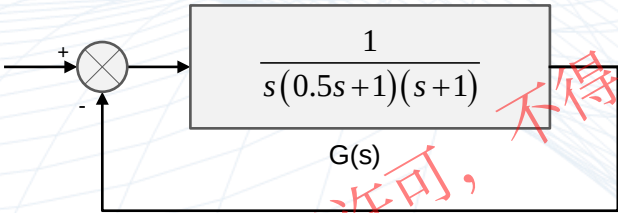
②. 闭环频域指标



闭环频域指标	含义
闭环谐振频率 ω_r	产生谐振峰值对应的频率
闭环谐振峰值 M_r	谐振频率处对应的幅值大小
带宽 ω_b	对数幅频特性的幅值下降到-3dB时对应的频率

通过 `getPeakGain()` 、 `bandwidth()` 函数计算系统闭环谐振峰值与带宽

示例3.13：考虑如图闭环系统，利用Syslab求取其闭环表达式，并计算其谐振峰值、谐振频率和带宽



```
s = tf('s')
Gopenloop = 1/(s*(0.5*s+1)*(s+1))
Gcloseloop = feedback(Gopenloop)

gpeak, fpeak = getPeakGain(Gcloseloop)
fb = bandwidth(Gcloseloop)
```

输出

```
# 以绝对单位给出谐振峰值，可通过
# 20*log10.(gpeak)计算dB值
julia> gpeak, fpeak =
getPeakGain(Gcloseloop)
(1.8371191411343013, 0.8164820126772819)

julia> fb = bandwidth(Gcloseloop)
1x1 Matrix{Float64}:
 1.2697224015878297
```

考虑通过带宽含义计算得到系统带宽

```
# 给定频率向量
w = logspace(-1,1,500)
# 定义系统模型
s = tf('s')
Gopenloop = 1/(s*(0.5*s+1)*(s+1))
Gcloseloop = feedback(Gopenloop)
# 计算频率响应
mag,phase,w = bode(Gcloseloop,w,fig = false)
# 计算系统带宽
n = 1
while 20*log10(mag[n]) >= -3
    n = n + 1
end
# 获取带宽值
w[n]
```

```
julia> w[n]
1.2653317593889428
```

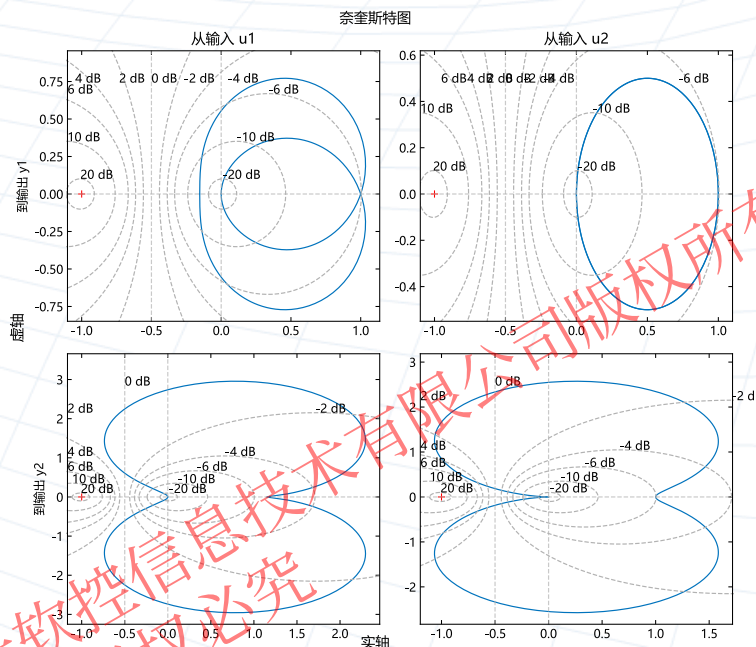
3.2 线性控制系统分析 – 频域分析

二. 奈奎斯特图 – nyquist()

示例3.14：计算示例3.11的MIMO系统频率响应并绘制Nyquist图

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} -1 & -1 \\ 6.5 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$
$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

```
# 定义系统矩阵
A = [-1 -1; 6.5 0]
B = [1 1; 1 0]
C = [1 0; 0 1]
D = zeros(2,2)
# 创建状态空间模型
G = ss(A,B,C,D)
# 绘制系统Nyquist图
nyquist(G)
nyquistgrid(true)
```

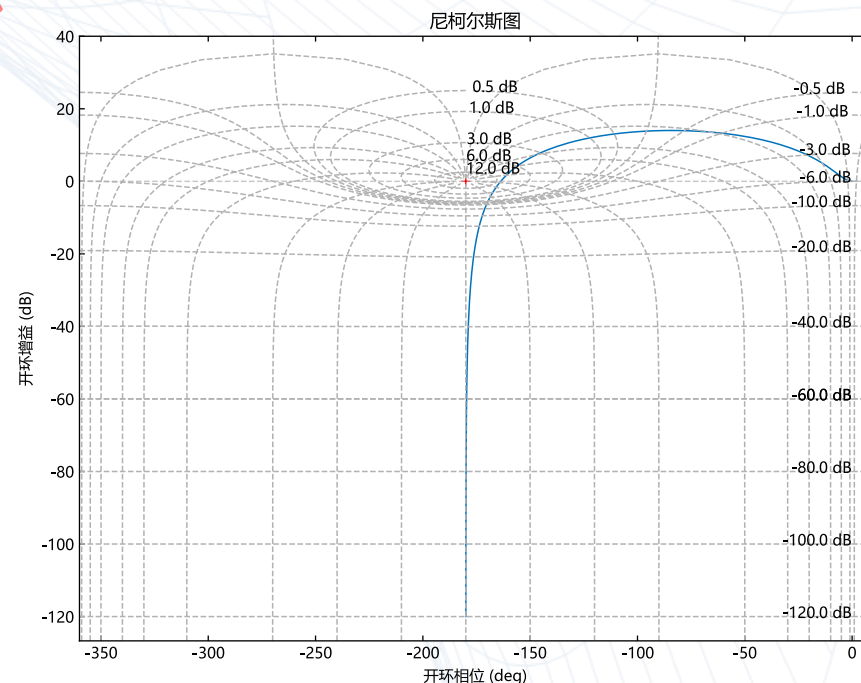


三. 尼克尔斯图 – nichols()

示例3.15：绘制系统对数幅-相图

$$G(s) = \frac{100}{s^2 + 2s + 100}$$

```
# 定义系统
G = tf([100],[1 2 100])
# 绘制nichols图并开启网格
nichols(G)
nicholsgrid(true)
```



Nyquist图、Nichols图的绘制、修饰与bode()用法一致，具体可以参考bode图示例及参考文档，这里不再赘述

3.2 线性控制系统分析 – 根轨迹

根轨迹法是经典控制理论的两大核心设计方法之一（另一为频率响应法）

根轨迹定义：当系统开环增益 K 由 0 到 ∞ 变化时，闭环极点在 s 平面的轨迹

根轨迹法的理论基础：闭环系统瞬态响应的基本特性与闭环极点的位置紧密相关。如果系统具有可变的开环增益，则闭环极点的位置取决于所选择的开环增益值。因此，当开环增益变化时，系统闭环极点在 s 平面如何移动，以及如何设计校正环节使得闭环极点按照期望的位置移动，即是根轨迹设计方法的基本原理。

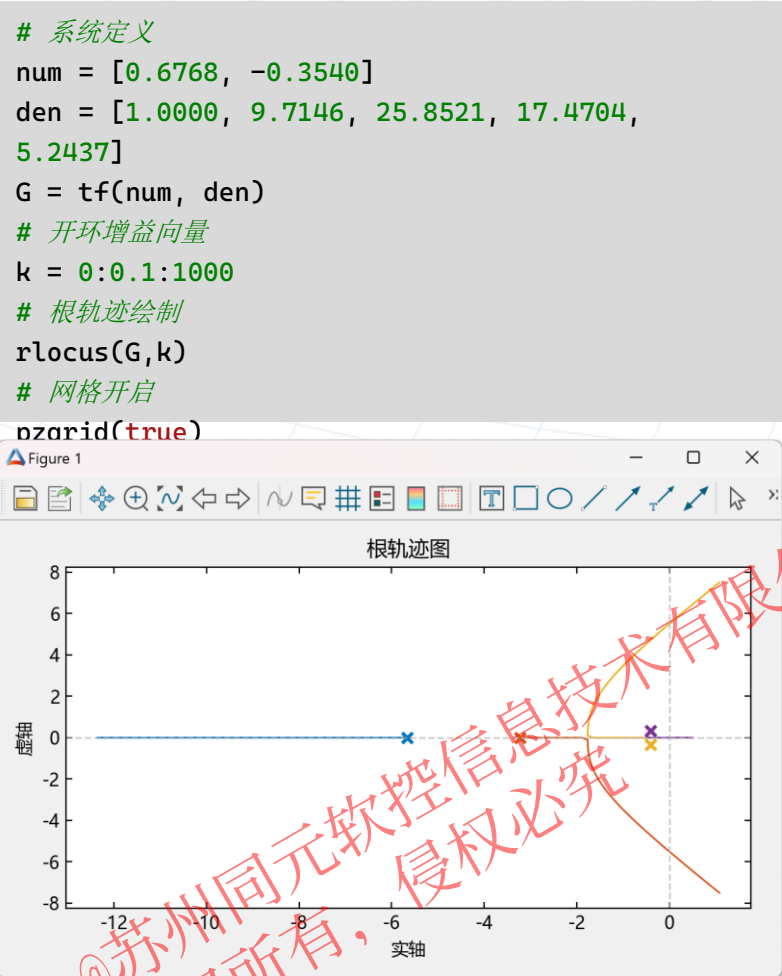
求解三阶以上的特征方程根异常麻烦，Syslab则为该问题提供了一个简便的解法

rlocus 调用方式	说明
rlocus(sys)	计算系统根轨迹，并给出根轨迹图，支持时间连续系统与时间离散系统
rlocus(sys, K)	指定系统开环增益K，计算系统根轨迹，并绘制系统根轨迹图 • K 为标量：开环增益计算范围为[0, K]，Syslab将自动计算增益向量 • K 为向量：系统将按照 K 计算指定增益的的根轨迹响应
z, p, k = rlocus(sys, fig = false)	计算系统根轨迹，不出图。其中 • z:系统零点 • p:闭环极点数组 • k: 开环增益数组
z, p, k = rlocus(sys, k, fig = false)	计算系统在指定开环增益K下的根轨迹，不出图。其中： • z:系统零点 • p:闭环极点数组 • k: 开环增益数组
pzgrid()	根轨迹网格开关。pzgrid(true)开启网格，pzgrid(false)关闭网格

3.2 线性控制系统分析 – 根轨迹

示例3.16：绘制以下系统根轨迹，考虑开环增益从0~1000变化

$$G(s) = \frac{0.6786s - 0.3540}{s^4 + 9.7146s^3 + 25.8521s^2 + 17.4704s + 5.2437}$$



闭环极点位置对系统性能的影响

系统的瞬态响应类型由闭环极点确定，一阶系统简单，其总是稳定的、无震荡、响应单调；而高阶系统的响应一般是由一阶和二阶系统响应的组合构成。所以这里主要讨论闭环极点与二阶系统时域响应性能关系。

一般情况下，针对高阶系统进行控制律设计时，常常需要对高阶系统的增益进行调整，以便使系统具有一对闭环主导共轭复数极点，此时，闭环极点将起到相对主导作用。如果实部的比值超过5，且在极点附近不存在零点，那么距离s平面虚轴最近的闭环极点将对瞬态响应起主导作用。在这种情况下，高阶系统可以近似考虑仅由其主导极点构成的二阶系统来近似。

考虑典型的二阶系统表达：

$$G(s) = \frac{Y(s)}{R(s)} = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

上述系统的特征方程：

$$s^2 + 2\zeta\omega_n s + \omega_n^2 = (s + p_1)(s + p_2) = 0$$

求解得到系统极点（特征根）：

$$\begin{cases} -p_1 = -\zeta\omega_n + \omega_n\sqrt{\zeta^2 - 1} \\ -p_2 = -\zeta\omega_n - \omega_n\sqrt{\zeta^2 - 1} \end{cases}$$

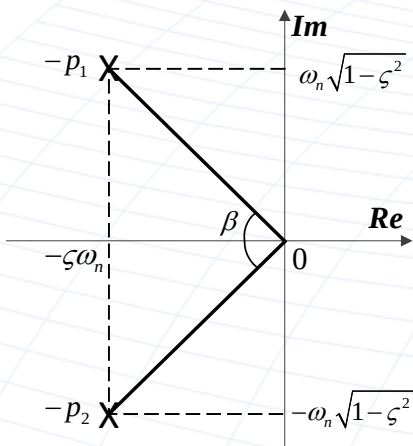
考虑不同阻尼系数对系统的影响

阻尼系数		特征根	特征根位置	响应曲线
无阻尼	$\zeta = 0$	$\pm j\omega_n$	虚轴上一对共轭虚根	等幅振荡
欠阻尼	$0 < \zeta < 1$	$-\zeta\omega_n \pm j \cdot \omega_n\sqrt{1 - \zeta^2}$	s左半平面的一对共轭复根	衰减震荡
临界阻尼	$\zeta = 1$	$-\omega_n$	负实轴上一对重根	单调上升
过阻尼	$\zeta > 1$	$-\zeta\omega_n \pm \omega_n\sqrt{\zeta^2 - 1}$	负实轴上两个互异根	单调上升

3.2 线性控制系统分析 – 根轨迹

闭环极点位置对系统性能的影响

针对典型二阶欠阻尼系统，其特征根为 s 左半平面一对共轭复根



β 为二阶系统共轭复根对负实轴的张角，显然有以下的关系： $\zeta = \cos \beta$

因此， β 被称为阻尼角，系统阻尼越大，阻尼角越小

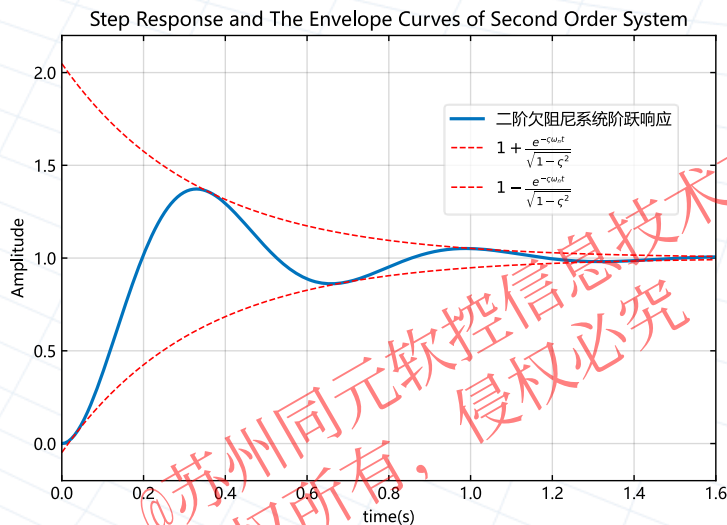
根据阶跃响应时域表达式可以计算系统调整时间，调整时间是指系统响应维持在稳态值的某个误差百分比 δ 范围内所需要的时间。考虑误差为 $\pm 2\%$ 的系统调整时间：

$$e^{-\zeta\omega_n T_s} < 0.02$$

得到调整时间近似值：

$$T_s = \frac{4}{\zeta\omega_n}$$

因此，调整时间与极点实部的绝对值 $|\zeta\omega_n|$ 呈反比



基于根轨迹法的控制器设计，可依据上述分析作为依据参考，结合Syslab所绘制的系统根轨迹，辅助开展校正环节的设计。

3.2 线性控制系统分析 – 能观能控性分析

Syslab提供一系列函数支持系统的能观能控性分析

系统特性计算或分析函数	说明
ctrb	状态空间模型的可控性
ctrbf	计算可控性阶梯形式
obsv	状态空间模型的可观察性
obsvf	计算可观察阶梯形式
lyap	连续李亚普诺夫方程解
dlyap	求解离散时间李亚普诺夫方程
icare	连续时间代数Riccati方程的隐式求解器
idare	离散时间代数Riccati方程的隐式求解器
norm	线性模型的范数
gram	Gramians方法判断可控性和可观察性

这些函数用法比较简单，因此不再举例说明，详细用法可以参考Syslab控制工具箱帮助文档

04

PID控制器设计

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

4.1 PID控制在工业界的应用



国际自动控制联合会（IFAC）每三年召开一次的世界大会是自动控制领域的顶级学术会议

TABLE 1 A list of the survey results in order of industry impact as perceived by the committee members.

Rank and Technology	High-Impact Ratings	Low- or No-Impact Ratings
PID control	100%	0%
Model predictive control	78%	9%
System identification	61%	9%
Process data analytics	61%	17%
Soft sensing	52%	22%
Fault detection and identification	50%	18%
Decentralized and/or coordinated control	48%	30%
Intelligent control	35%	30%
Discrete-event systems	23%	32%
Nonlinear control	22%	35%
Adaptive control	17%	43%
Robust control	13%	43%
Hybrid dynamical systems	13%	43%

IFAC在2014年发布的一项调查报告显示，PID仍然是在工业领域具有最大影响力的控制技术

Tariq Samad, A Survey on Industry Impact and Challenges Thereof

<https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=7823045>

Table 2

The percentage of survey respondents indicating whether a control technology had demonstrated (“Current Impact”) or was likely to demonstrate over the next five years (“Future Impact”) high impact in practice.

Control Technology	Current Impact	Future Impact
PID control	91%	78%
System Identification	65%	72%
Estimation and filtering	64%	63%
Model-predictive control	62%	85%
Process data analytics	51%	70%
Fault detection and identification	48%	78%
Decentralized and/or coordinated control	29%	54%
Robust control	26%	42%
Intelligent control	24%	59%
Discrete-event systems	24%	39%
Nonlinear control	21%	42%
Adaptive control	18%	44%
Repetitive control	12%	17%
Hybrid dynamical systems	11%	33%
Other advanced control technology	11%	25%
Game theory	5%	17%

2020年在Annual Reviews in Control中发表的Industry engagement with control research: Perspective and messages, 表明PID控制的当前影响力最高，且在未来5年内可能是工业实践中影响力较高的控制技术，紧随其后的则是MPC技术

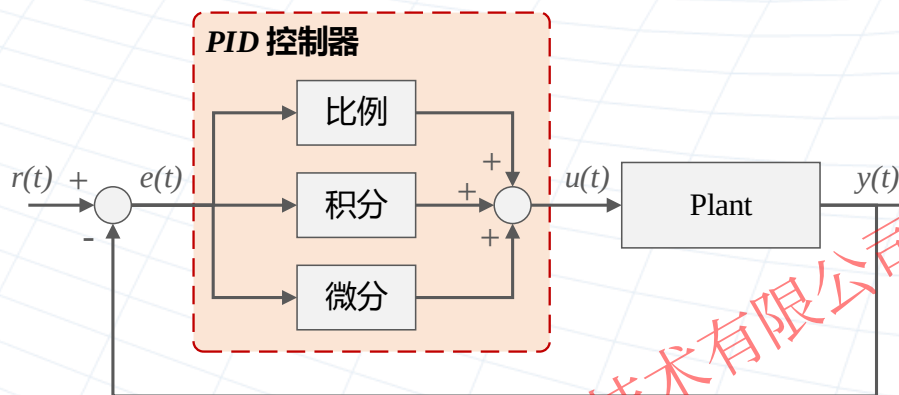
<https://www.sciencedirect.com/science/article/pii/S1367578820300080>



4.2 PID控制器原理分析

在当今应用的工业控制器中，有半数以上采用了PID或变形PID控制器。PID控制的价值取决于它们对大多数控制系统的广泛适用性，体现在：

- **原理简单**，应用方便，参数整定灵活
- **适用性强**，特别是当被控对象的数学模型未知，而不能使用解析设计方法时，PID控制就显得特别有用
- **鲁棒性强**，控制品质对受控对象的变化不太敏感，如受控对象受外界扰动时，无需经常改变控制器参数或结构



PID控制是通过误差信号 $e(t)$ 进行比例、积分、微分运算和结果的加权处理，得到控制器的输出 $u(t)$ ，最为控制对象的控制值。

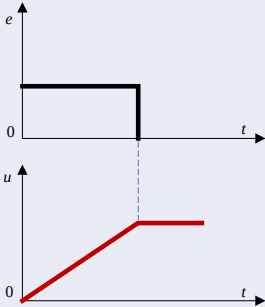
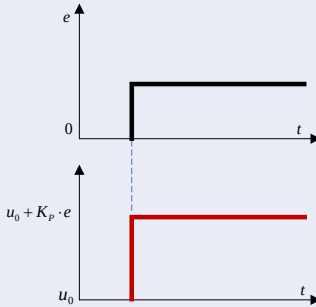
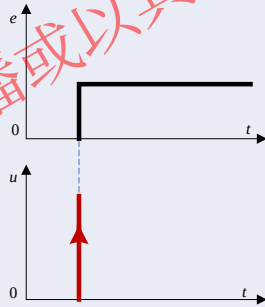
$$u(t) = K_p \cdot e(t) + \frac{1}{K_I} \int_0^t e(t) dt + K_D \frac{de(t)}{dt}$$

经Laplace变换后，PID控制器可描述为：

$$G_c(s) = K_p + \frac{1}{K_I s} + K_D s$$

其中， K_p 为比例系数， K_I 为积分时间常数， K_D 微分时间常数

4.2 PID控制器原理分析

	积分控制	比例控制	微分控制
控制作用与表达式	 $u = \frac{1}{K_I} \cdot \int_0^t e \, dt$	 $u = K_P \cdot e$	 $u = K_D \cdot \frac{de}{dt}$
特点	- 控制输出不仅与偏差大小有关，还与偏差存在的时间长短有关 - 无差调节。只要偏差存在，调节器就会一直作用，直到偏差为0，调节器会稳定不变	- 偏差一旦发生，即时控制，没有时滞，动态性好 - 有差调节。被调节量无法与设定值完全相等，它们之间一定有残差	- 根据误差变化率采取控制作用，偏差变化越剧烈，控制作用越强，能够提升系统稳定性，偏差无变化，则不起作用 - 有差调节。微分调节无法完全消除残差
系数影响	$S = 1/K_I$ 为积分速度 增大积分速度：能够加快调节速度，但当积分速度大于某一临界值后，系统可能不稳定。另外，积分速度越快，越容易引起震荡 减小积分速度：偏差消除速度减慢，系统稳定性增加 一般不会单独使用，其作为一种辅助的调节，通常与比例控制组合使用	增加比例系数：实质上相当增加开环放大倍数，系统响应加快，稳态误差减小但不能消除，增大到一定阈值后，系统可能会发散不稳定 减小比例系数：响应减慢，稳态误差变大，但稳定裕度提升	增大微分系数：控制作用增强。需要说明的是，微分作用与偏差大小无关，与偏差变化率正相关 减小微分系数：控制作用减小。

积分控制，对过去的偏差进行累计并消除

不忘历史

比例控制，对现在的偏差进行即时调整

把握当下

微分控制，预测将来的偏差并作出反应

展望未来

4.2 PID控制器原理分析 – 纯比例控制

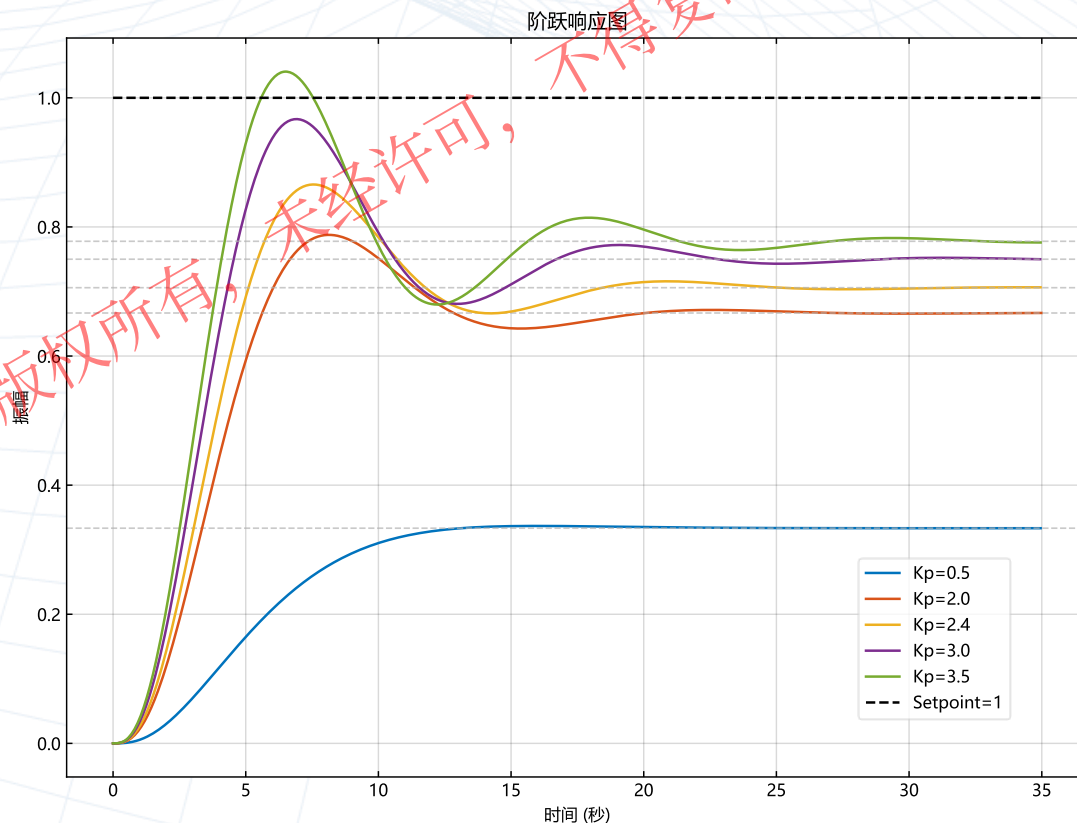
Syslab提供 `pid()` 函数支持创建并行形式 (parallel-form) 的连续时间、离散时间PID控制器

示例4.1: 已知被控对象传递函数如下，考虑采用纯比例控制，比例系数分别为： $K_p = 0.5、2.0、2.4、3.0、3.5$ ，求系统闭环的单位阶跃响应

$$C_p(s) = K_p, K_p = [0.5, 2.0, 2.4, 3.0, 3.5]$$

$$G(s) = \frac{1}{(s+1)(2s+1)(5s+1)}$$

```
# 创建被控对象模型
s = tf("s")
G = 1/((s+1)*(2*s+1)*(5*s+1))
Kp = [0.5, 2.0, 2.4, 3.0, 3.5]
# 预定义开环系统
GCp = Array{TyControlSystems.TransferFunction}{undef,
length(Kp)}
# 不同增益的开环系统定义及闭环阶跃响应计算
for i in 1:length(Kp)
    GCp[i] = pid(Kp[i])*G
    step(feedback(GCp[i]),35,ishold = true,linewidth = 1.5)
end
grid("on")
hold("on")
# SetPoint绘制
plot([0,35],[1,1],"--k",linewidth = 1.5)
# legend绘制
lines = gca().get_lines()
legend([lines[1], lines[3], lines[5], lines[7], lines[9],
lines[11]], ["Kp=0.5", "Kp=2.0", "Kp=2.4", "Kp=3.0", "Kp=3.5",
"Setpoint=1"])
```



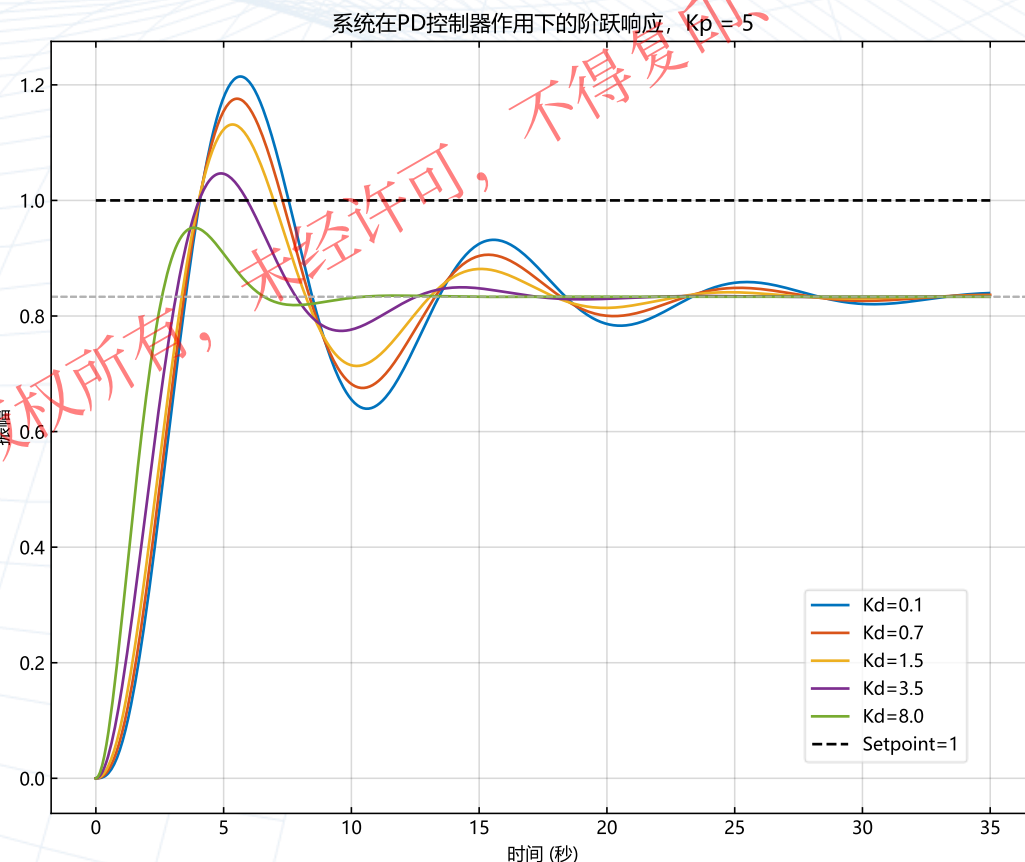
- 纯P控制是有差调节
- 比例系数越大，系统稳态误差越小，响应越快，但超调越大

4.2 PID控制器原理分析 – 比例-微分控制

示例4.2：对上述示例的被控对象采用PD控制，确定比例系数为 $K_p = 5$ ，考虑微分系数分别为： $K_d = 0.1、0.7、1.5、3.5、8.0$ ，求系统闭环的单位阶跃响应

$$C_{pd}(s) = K_p + K_d \cdot s, \quad K_p = 5, K_d = [0.1, 0.7, 1.5, 3.5, 8.0]$$

```
# 创建被控对象模型
s = tf('s')
G = 1/((s+1)*(2*s+1)*(5*s+1))
Kd = [0.1, 0.7, 1.5, 3.5, 8.0]
# 预定义开环系统
GCpd = Array{TyControlSystems.TransferFunction}(undef,
length(Kd))
# 不同增益的开环系统定义及闭环阶跃响应计算
for i in 1:length(Kd)
    GCpd[i] = pid(5, 0, Kd[i])*G
    step(feedback(GCpd[i]), 35, ishold = true, linewidth = 1.5)
end
grid("on")
hold("on")
title("系统在PD控制器作用下的阶跃响应, Kp = 5")
# SetPoint绘制
plot([0, 35], [1, 1], "--k", linewidth = 1.5)
# legend绘制
lines = gca().get_lines()
legend([lines[1], lines[3], lines[5], lines[7], lines[9],
lines[11]], ["Kd=0.1", "Kd=0.7", "Kd=1.5", "Kd=3.5", "Kd=8.0",
"Setpoint=1"])
```



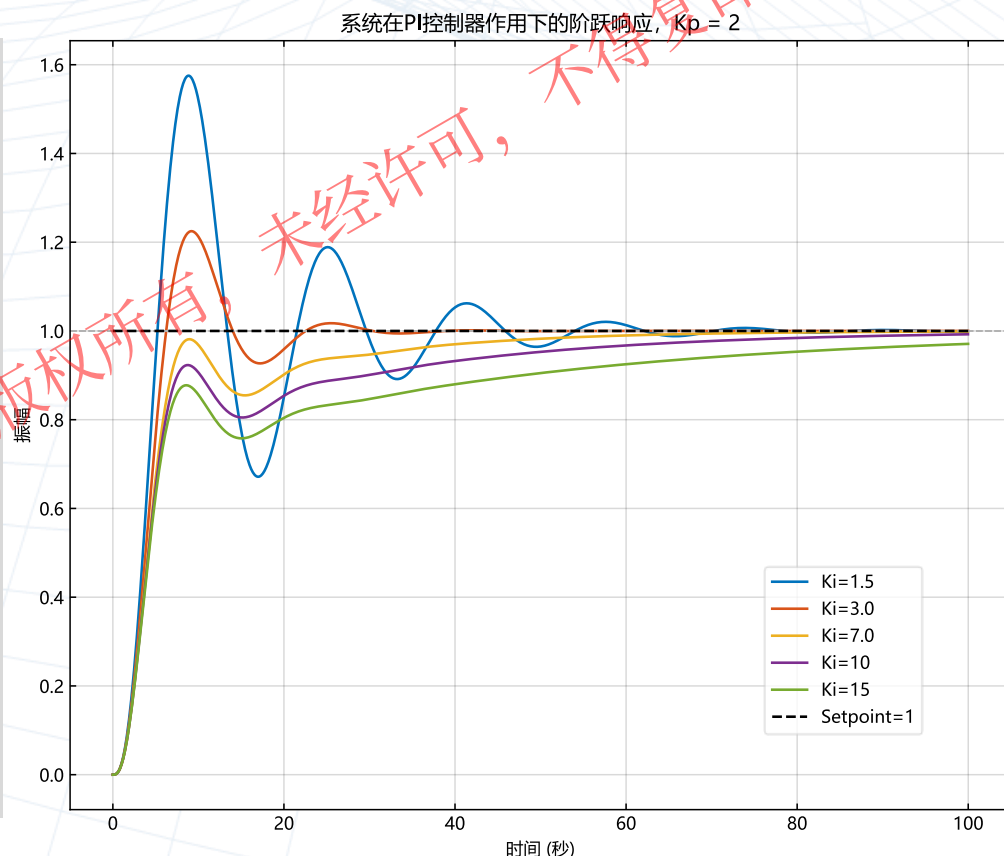
- PD控制引入微分项后，提升了系统稳定性，且加快了系统响应速度
- 但PD控制无法消除系统残差

4.2 PID控制器原理分析 – 比例-积分控制

示例4.3：对上述示例的被控对象采用PI控制，确定比例系数为 $K_p = 2$ ，考虑积分系数分别为： $K_i = 1.5、3.0、7.0、10、15$ ，求系统闭环的单位阶跃响应

$$C_{pi}(s) = K_p + \frac{1}{K_i \cdot s}, \quad K_p = 2, K_i = [1.5, 3.0, 7.0, 10, 15]$$

```
# 创建被控对象模型
s = tf("s")
G = 1/((s+1)*(2*s+1)*(5*s+1))
Ki = [1.5, 3, 7, 10, 15]
# 预定义开环系统
GCpi = Array{TyControlSystems.TransferFunction}(undef,
length(Kd))
# 不同增益的开环系统定义及闭环阶跃响应计算
for i in 1:length(Kd)
    GCpi[i] = pid(2,1/Ki[i])*G
    step(feedback(GCpi[i]),100,ishold = true,linewidth = 1.5)
end
grid("on")
hold("on")
title("系统在PI控制器作用下的阶跃响应, Kp = 2")
# SetPoint绘制
plot([0,100],[1,1],"--k",linewidth = 1.5)
# legend绘制
lines = gca().get_lines()
legend([lines[1], lines[3], lines[5], lines[7], lines[9],
lines[11]], ["Ki=1.5", "Ki=3.0", "Ki=7.0", "Ki=10", "Ki=15",
"Setpoint=1"])
```



- PI控制器引入了积分项，可以消除残差
- 本材料中给出的积分系数为 $1/K_i$ ，因此， K_i 越小，积分作用越强，系统稳定性会越差

4.2 PID控制器原理分析 – 比例-积分-微分控制

示例4.4: 针对上述系统，对比采用P控制、PD控制、PI控制，PID控制的响应情况

```
# 创建被控对象模型
s = tf("s")
G = 1/((s+1)*(2*s+1)*(5*s+1))
# 定义各类型PID控制器
C = [pid(3.5),pid(3.5,0,3.5),pid(3.5,1/2),pid(3.5,1/2,3.5)]
# 预定义开环系统
GC = Array{TyControlSystems.TransferFunction}(undef,
length(C))
# 开环系统定义及闭环阶跃响应计算
for i in 1:length(C)
    GC[i] = C[i]*G
    step(feedback(GC[i]),60,ishold = true,linewidth = 1.5)
end
grid("on")
hold("on")
title("系统在各类PID控制器作用下的阶跃响应")
# SetPoint绘制
plot([0,60],[1,1],"--k",linewidth = 1.5)
# legend绘制
Cp = raw"$C_{P} = 3.5$"
Cpd = raw"$C_{PD} = 3.5 + 3.5s$"
Cpi = raw"$C_{PI} = 3.5 + \frac{1}{2s}$"
Cpid = raw"$C_{PID} = 3.5 + \frac{1}{2s} + 3.5s$"
lines = gca().get_lines()
legend([lines[1], lines[3], lines[5], lines[7]],
[Cp,Cpd,Cpi,Cpid])
```

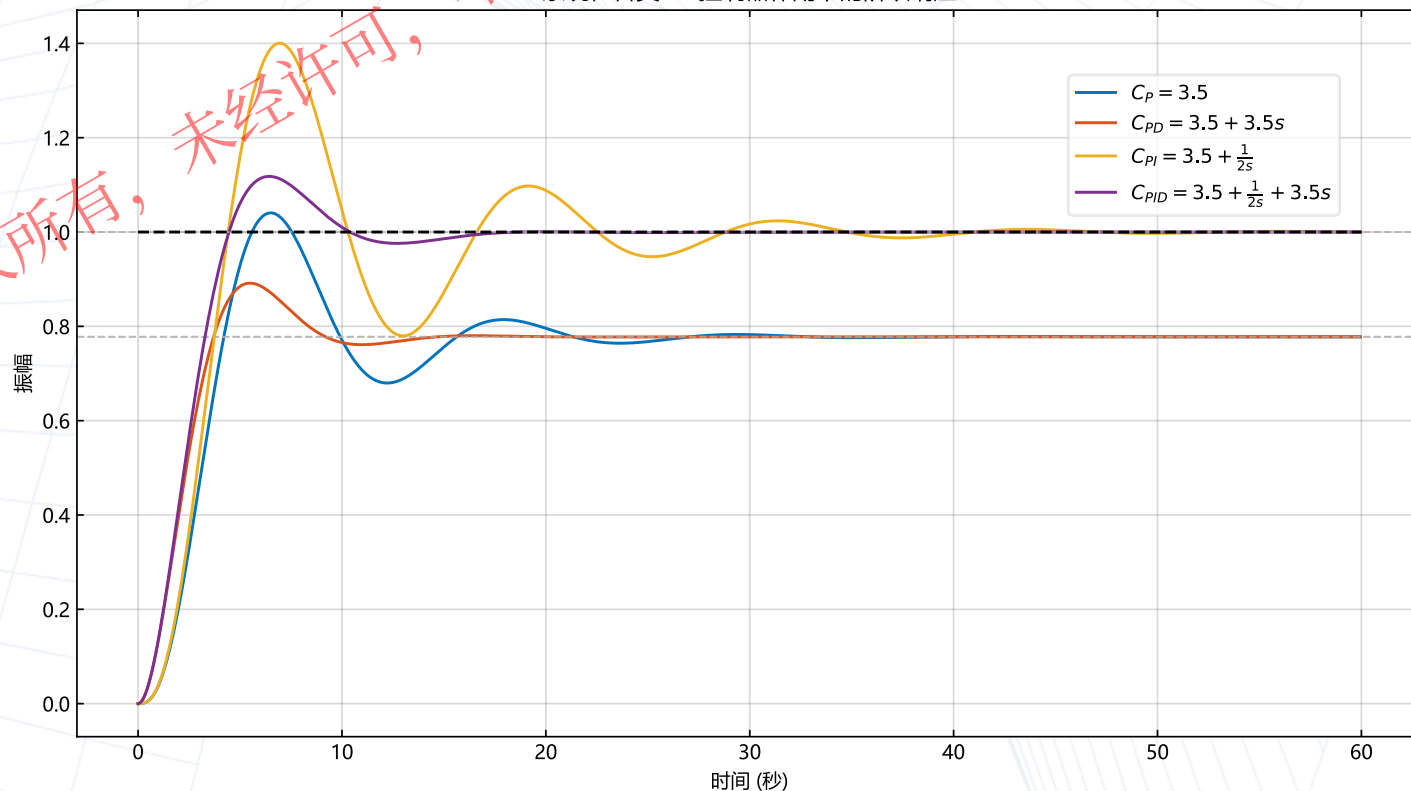
$$C_p = K_p, K_p = 3.5$$

$$C_{pd} = K_p + K_d \cdot s, K_p = 3.5, K_d = 3.5$$

$$C_{pi}(s) = K_p + \frac{1}{K_i \cdot s}, K_p = 3.5, K_i = 2$$

$$C_{pid}(s) = K_p + \frac{1}{K_i \cdot s} + K_d \cdot s, K_p = 3.5, K_i = 2, K_d = 3.5$$

系统在各类PID控制器作用下的阶跃响应



4.3 PID控制器设计

本章节介绍了PID控制器原理，并通过示例演示了各个调节的作用、以及P、I、D控制器的组合作用

实际上，在工业中PID控制器的设计有不少经验与解析的方法，这些方法均以设计指标为输入，通过一定的计算流程，得到PID控制器参数
比如：

- 齐格勒-尼科尔斯法 (Ziegler–Nichols) 设计PID控制器
- 频率响应法设计PID控制器

这些方法均为成熟方法，利用Syslab控制系统工具箱提供的函数，均可以实现上述方法的PID控制器设计，这里不再赘述。

有兴趣尝试的同学，可参考：《Modern Control Engineering》Fifth Edition, Katsuhiko Ogata

05

状态反馈控制律设计

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司
版权所有，侵权必究

5.1 状态空间设计-极点配置方法

极点配置：设计控制器使闭环系统的极点位于所希望的极点位置

极点配置方法类似于根轨迹法，但他们的基本区别是，根轨迹法仅配置了主导闭环极点，而极点配置设计则把所有的闭环极点都配置到希望的位置。

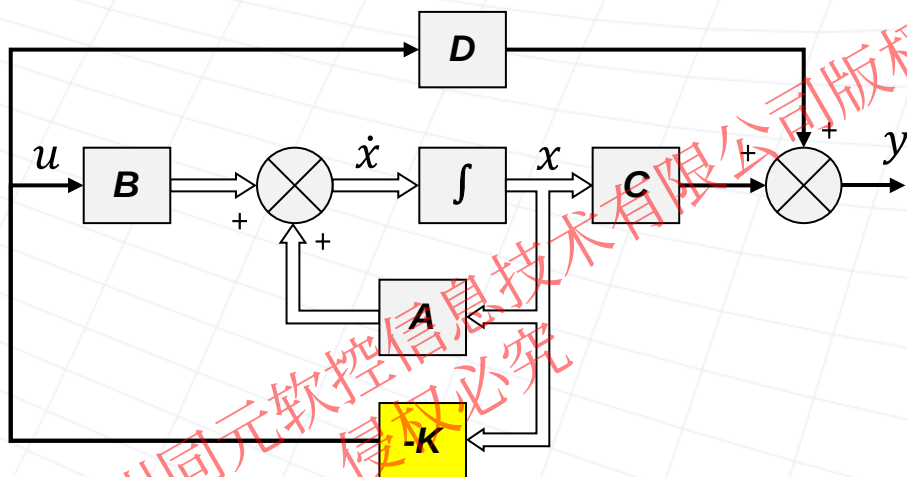
考虑SISO控制系统状态空间表达式如下：

$$\dot{x} = Ax + Bu$$

$$y = Cx + Du$$

其中， x 为状态向量（ n 维）， y 为输出信号（标量）， u 为控制信号（标量）， A 为 $n \times n$ 维定常矩阵， B 为 $n \times 1$ 维定常矩阵， C 为 $1 \times n$ 维定常矩阵， D 为常量

假设所期望的闭环极点位于 $s_1 = \mu_1, s_2 = \mu_2, \dots, s_n = \mu_n$ 。如果原系统是状态完全可控的（充要条件），那么可以通过选取一个适当的状态反馈增益 K ，使系统具有的闭环极点位于期望的位置上



$u = -Kx$ 时的闭环控制系统

下图的闭环系统无输入量，其目的是保持系统输出为零。这个系统称之为调节器系统。

将 $u = -Kx$ 带入原始系统，得到系统闭环方程：

$$\dot{x}(t) = (A - BK) \cdot x(t)$$

该方程的解是：

$$x(t) = e^{(A-BK)t} x(0)$$

其中 $x(0)$ 是外部扰动引起的初始状态，系统的稳态响应与瞬态响应特性由矩阵 $A - BK$ 的特征值决定。选择合适的矩阵 K ，可使矩阵 $A - BK$ 构成一个渐进稳定矩阵，并针对所有的 $x(0) \neq 0$ ，当 t 趋近无穷，都可使 $x(t)$ 趋于0。称矩阵 $A - BK$ 的特征值为调节器的极点。这就是极点配置问题。

$$|\lambda I - (A - BK)| = \prod_{i=1}^n (s - \mu_i)$$

求解上述方程，得到的 K 即为状态反馈增益。Syslab提供 `acker()`、`place()` 函数以支持极点配置的求解

5.2 基于极点配置法的I型伺服系统设计 – 控制对象含有一个积分器的情况

上一节讨论的是调节器系统，其无输入量，目标是保持系统输出为零。但伺服系统的设计目标则是：寻找合适的控制量，让系统输出能够跟随给定值（输入量）的变化。

考虑含有一个积分器的SISO控制系统状态空间

表达式如下：

$$\dot{x} = Ax + Bu$$

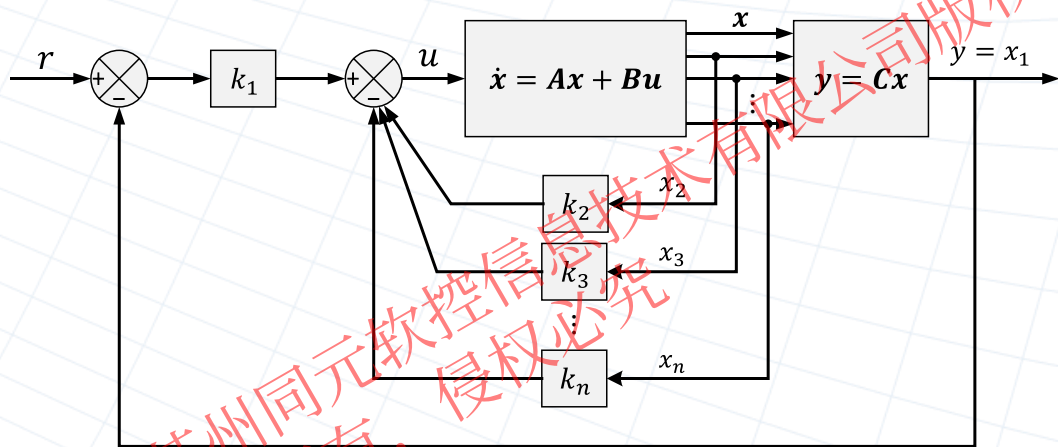
$$y = Cx$$

其中， x 为状态向量（ n 维）， y 为输出信号（标量）， u 为控制信号（标量）， A 为 $n \times n$ 维定常矩阵， B 为 $n \times 1$ 维定常矩阵， C 为 $1 \times n$ 维定常矩阵，假设 $y = x_1$

设定系统参考输入为 r ，在该系统中，采用以下状态反馈控制方案：

$$u = -\begin{bmatrix} 0 & k_2 & k_3 & \cdots & k_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} + k_1(r - x_1) = -Kx + k_1r$$

$$K = \begin{bmatrix} k_1 & k_2 & k_3 & \cdots & k_n \end{bmatrix}$$



将 $u = -Kx + k_1r$ 带入系统方程形成闭环系统：

$$\dot{x} = Ax + Bu = (A - BK)x + Bk_1r$$

系统在稳态时：

$$\dot{x}(\infty) = (A - BK)x(\infty) + Bk_1r(\infty) \quad r \text{ 为输入量, } r(\infty) = r(t)$$

上述两个方程作差得到：

$$\dot{x}(t) - \dot{x}(\infty) = (A - BK)[x(t) - x(\infty)]$$

令 $x(t) - x(\infty) = e(t)$ ， $e(t)$ 则描述了系统的误差动态特性

$$\dot{e}(t) = (A - BK) \cdot e(t)$$

注意：这里I型伺服系统的设计转换成了在给定任意初始条件 $e(0)$ 下，使 $e(t)$ 趋于零的渐进稳定调节器系统设计。如果原始系统完全可控，那么在指定了矩阵 $A - BK$ 的希望特征值 $s_1 = \mu_1, s_2 = \mu_2, \dots, s_n = \mu_n$ 时，利用前节介绍的极点配置方法即可求解矩阵 K 。

此时，即可利用Syslab提供 `acker()`、`place()` 函数进行极点配置求解

5.2 基于极点配置法的I型伺服系统设计 – 控制对象含有一个积分器的情况

示例5.1：当控制对象传递函数具有一个积分器时，试设计一个I型伺服系统。设控制对象传递函数为：

$$\frac{Y(s)}{U(s)} = \frac{1}{s(s+1)(s+2)}$$

希望的闭环极点为： $s = -2 \pm j \cdot 2\sqrt{3}$ ，和 $s = -10$ 。设计系统状态反馈增益 K ，并计算系统单位阶跃响应

①. 被控对象定义及模型转化

```
# 定义被控对象
s = tf("s")
G = 1/(s*(s+1)*(s+2))
# 将被控对象转化为状态空间模型
Gss = ss(G)
```

```
julia> Gss
A =
 0.0  1.0  0.0
 0.0  0.0  1.0
 0.0 -2.0 -3.0
B =
 0.0
 0.0
 1.0
C =
 1.0  0.0  0.0
D =
 0.0

连续时间状态空间模型
```

②. 定义期望极点位置并计算状态反馈增益K

```
# 定义闭环系统期望极点
idealpoles = [-2+im*2*sqrt(3),-2-im*2*sqrt(3),-10]
# 使用place函数求解状态反馈增益
K = place(Gss.A,Gss.B,idealpoles)
K = real(K)
```

```
julia> K
1x3 Matrix{ComplexF64}:
 160.0+0.0im  54.0+0.0im  11.0+0.0im
```

带入求取得到的状态反馈增益矩阵 K ，系统闭环方程为：

$$\dot{x} = (A - BK)x + Bk_r$$

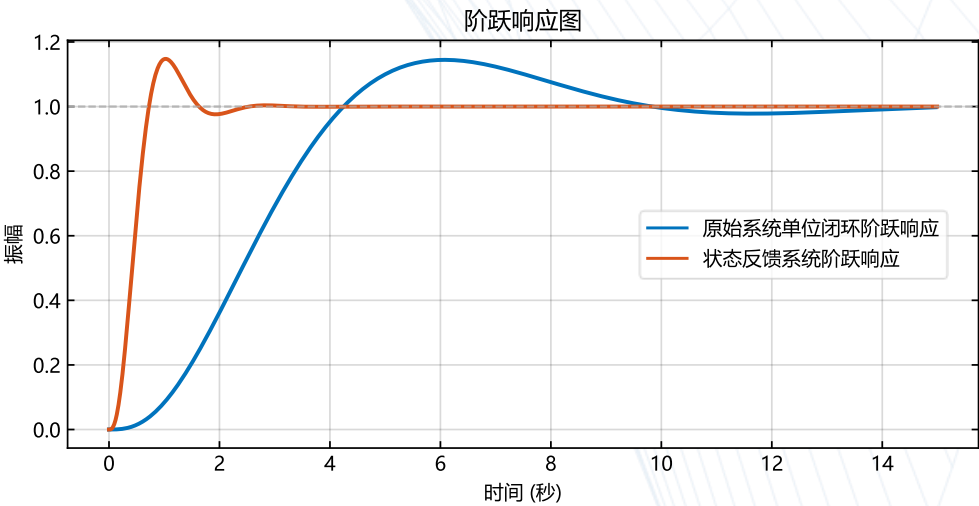
$$y = Cx$$

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -160 & -56 & -14 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 160 \end{bmatrix} \cdot r$$

$$y = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

③. 状态反馈闭环系统构造与阶跃响应计算

```
# 加入状态反馈的闭环系统计算
AA = Gss.A - Gss.B * K
BB = Gss.B * K[1]
CC = Gss.C
DD = Gss.D
# 被控对象单位闭环阶跃响应与状态反馈闭环阶跃计算对比
step(feedback(G),15,linewidth = 2)
step(ss(AA,BB,CC,DD),15,linewidth = 2,ishold = true)
grid("on")
```



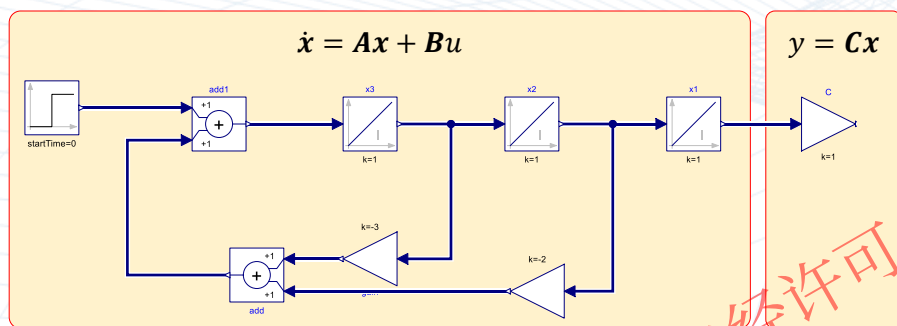
5.2 基于极点配置法的I型伺服系统设计 – 控制对象含有一个积分器的情况

示例5.2：使用Sysplorer的基本blocks实现示例XX的模拟结构图模型，并执行仿真验证（回应B站教学视频中关于极点配置如何计算）

https://www.bilibili.com/video/BV1zf4y1X7eA?spm_id_from=333.999.0.0&vd_source=47a11ba45a018ce644b28fd69319c3ec

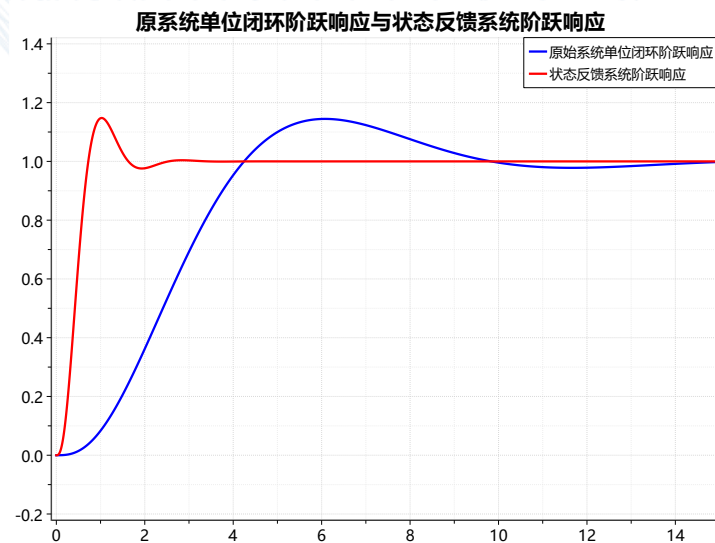
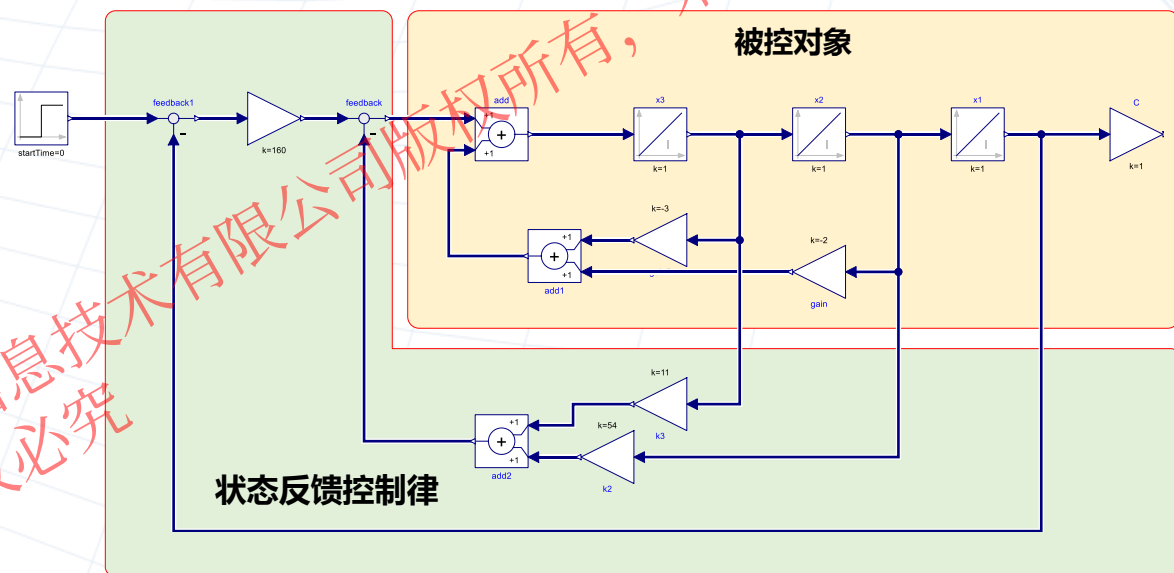
①. 被控对象动态仿真模型构建

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & -2 & -3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \cdot r$$
$$y = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$



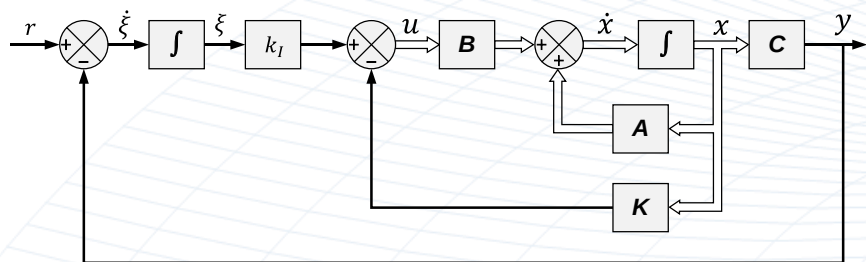
②. 状态反馈系统模型构建

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -160 & -56 & -14 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 160 \end{bmatrix} \cdot r$$
$$y = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$



5.3 基于极点配置法的I型伺服系统设计 – 控制对象无积分器的情况

如果控制对象无积分器（0型控制对象），I型伺服系统的基本设计原则是：在控制对象与误差比较器之前的前向通路中插入一个积分器，如下：



$$\begin{aligned}\dot{x} &= Ax + Bu \\ y &= Cx \\ u &= -Kx + k_I \xi \\ \xi &= r - y = r - Cx\end{aligned}$$

其中， x 为状态向量（ n 维）， y 为输出信号（标量）， u 为控制信号（标量）， ξ 为积分器输出， r 为参考输入信号， A 为 $n \times n$ 维定常矩阵， B 为 $n \times 1$ 维定常矩阵， C 为 $1 \times n$ 维定常矩阵

上述系统写成矩阵方程形式：

$$\begin{bmatrix} \dot{x}(t) \\ \dot{\xi}(t) \end{bmatrix} = \begin{bmatrix} A & 0 \\ -C & 0 \end{bmatrix} \begin{bmatrix} x(t) \\ \xi(t) \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} u(t) + \begin{bmatrix} 0 \\ 1 \end{bmatrix} r(t)$$

稳态方程：

$$\begin{bmatrix} \dot{x}(\infty) \\ \dot{\xi}(\infty) \end{bmatrix} = \begin{bmatrix} A & 0 \\ -C & 0 \end{bmatrix} \begin{bmatrix} x(\infty) \\ \xi(\infty) \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} u(\infty) + \begin{bmatrix} 0 \\ 1 \end{bmatrix} r(\infty)$$

问题转化为：设计一个渐进稳定系统，使得 $x(\infty)$ 、 $\xi(\infty)$ 和 $u(\infty)$ 分别趋于定常值，因此在稳态时， $\dot{\xi}(t) = 0$ ，且 $y(\infty) = r$

方程作差得到：

$$\begin{bmatrix} \dot{x}(t) - \dot{x}(\infty) \\ \dot{\xi}(t) - \dot{\xi}(\infty) \end{bmatrix} = \begin{bmatrix} A & 0 \\ -C & 0 \end{bmatrix} \begin{bmatrix} x(t) - x(\infty) \\ \xi(t) - \xi(\infty) \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} [u(t) - u(\infty)]$$

定义：

$$\begin{aligned}x(t) - x(\infty) &= x_e(t) \\ \xi(t) - \xi(\infty) &= \xi_e(t) \\ u(t) - u(\infty) &= u_e(t)\end{aligned}$$

方程可写成：

$$\begin{bmatrix} \dot{x}_e(t) \\ \dot{\xi}_e(t) \end{bmatrix} = \begin{bmatrix} A & 0 \\ -C & 0 \end{bmatrix} \begin{bmatrix} x_e(t) \\ \xi_e(t) \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} u_e(t) \quad e(t) = \begin{bmatrix} x_e(t) \\ \xi_e(t) \end{bmatrix}$$

定义一个新的 $(n+1)$ 维误差向量 $e(t)$ ：

$$\begin{aligned}\dot{e} &= \hat{A}e + \hat{B}u_e \\ \hat{A} &= \begin{bmatrix} A & 0 \\ -C & 0 \end{bmatrix}, \hat{B} = \begin{bmatrix} B \\ 0 \end{bmatrix} \\ u_e &= -\hat{K}e, \hat{K} = [K \mid -k_I]\end{aligned}$$

将输入带入状态

误差方程：

$$\dot{e} = (\hat{A} - \hat{B}\hat{K})e$$

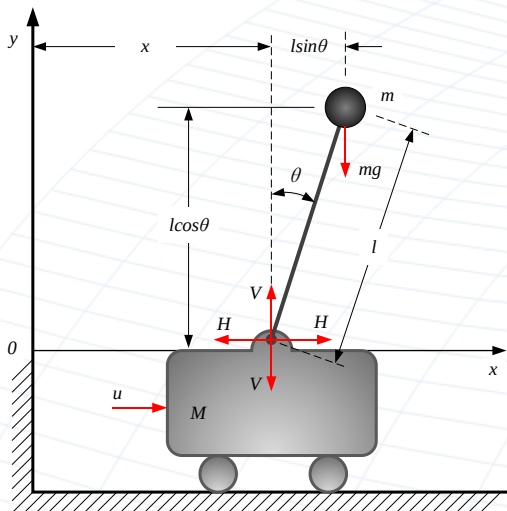
注意：希望矩阵 $\hat{A} - \hat{B}\hat{K}$ 的特征值（期望的闭环极点）为指定的 $\mu_1, \mu_2, \dots, \mu_{n+1}$ 时，则状态反馈增益 K 与积分增益 k_I ，在误差状态方程完全可控的条件下，可利用前节介绍的极点配置方法确定。

需要指出：矩阵 $\hat{A}$ 的秩为 $n+1$ ，则系统完全可控

5.3 基于极点配置法的I型伺服系统设计 – 控制对象无积分器的情况

示例5.3：一维倒立摆系统基于极点配置法的状态反馈控制律设计

①. 系统分析与数学建模



设计目标希望尽可能把倒立摆保持在垂直位置上，倒立摆安装在小车上，它没有积分器。忽略摆杆质量与地面摩擦，考虑其质量集中在摆杆顶端，即摆杆的重心就是摆球的重心

□ 考虑摆杆重心的水平运动：

$$m \frac{d^2}{dt^2} (x + l \sin \theta) = H$$

□ 摆杆重心的垂直运动：

$$m \frac{d^2}{dt^2} (l \cos \theta) = V - mg$$

□ 小车水平运动：

$$M \frac{d^2 x}{dt^2} = u - H$$

□ 再考虑摆杆转动，其绕重心（摆球重心）的转动运动可以描述为（ I 为摆杆绕其中心的转动惯量）：

$$I \frac{d^2 \theta}{dt^2} = Vl \sin \theta - Hl \cos \theta$$

线性化假设：考虑由于必须保持倒立摆垂直，可以认为 θ 角很小，因此，可以进行以下近似， $\sin \theta \approx \theta$ 、 $\cos \theta \approx 1$ ；再考虑倒立摆围绕重心的转动惯量很小，可近似认为： $I \approx 0$

联立方程，得到：

$$\begin{cases} (M + m) \ddot{x} + ml \ddot{\theta} = u \\ ml^2 \ddot{\theta} + ml \ddot{x} = mgl \theta \end{cases}$$

定义状态变量：

$$\begin{cases} x_1 = \theta \\ x_2 = \dot{\theta} \\ x_3 = x \\ x_4 = \dot{x} \end{cases}$$

方程改写为：

$$\begin{cases} \dot{x}_1 = x_2 \\ \dot{x}_2 = \frac{M + m}{Ml} g x_1 - \frac{1}{Ml} u \\ \dot{x}_3 = x_4 \\ \dot{x}_4 = -\frac{m}{M} g x_1 + \frac{1}{M} u \end{cases}$$

描述系统行为的状态方程：

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \\ \dot{x}_4 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ \frac{M + m}{Ml} g & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ -\frac{m}{M} g & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} + \begin{bmatrix} 0 \\ -\frac{1}{Ml} \\ 0 \\ \frac{1}{M} \end{bmatrix} u$$
$$y = [0 \quad 0 \quad 1 \quad 0] \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

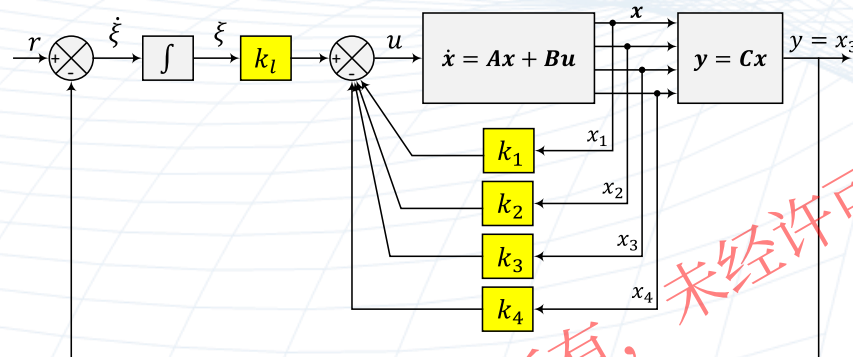
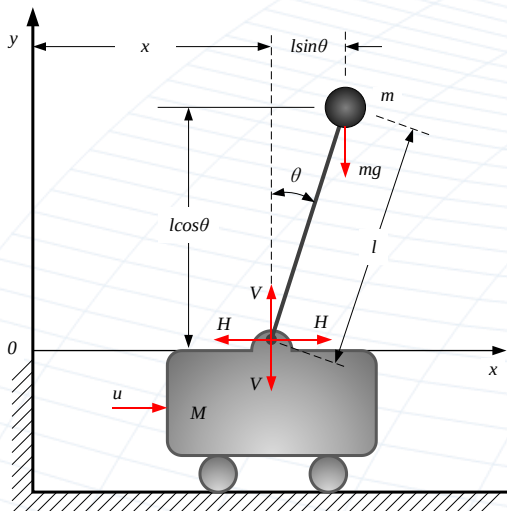
```
# 倒立摆系统参数
M = 2      # 小车质量2Kg
m = 0.1    # 摆球质量0.1Kg
l = 0.5    # 摆杆长度0.5m
g = 9.8    # 重力加速度9.8m/s^2
# 倒立摆系统状态空间矩阵定义
A = [0 1 0 0
      (M+m)*g/(M*l) 0 0 0
      0 0 0 1
      -m*g/M 0 0 0]
B = [0 -1 / (M * l) 0 1 / M]'
C = [0 0 1 0]
D = 0
# 倒立摆系统模型
G = ss(A, B, C, D)
```

5.3 基于极点配置法的I型伺服系统设计 – 控制对象无积分器的情况

示例5.3：一维倒立摆系统基于极点配置法的状态反馈控制律设计

②. 状态反馈控制律框架及推导

在被控对象与误差比较器之间的前向通路中插入一个积分器，
因此，整个系统的闭环框图为



整个系统的闭环表达式为：

$$\begin{cases} \dot{x} = Ax + Bu \\ y = Cx \\ u = -Kx + k_1 \xi \\ \dot{\xi} = r - Cx \end{cases} \Rightarrow \begin{bmatrix} \dot{x}(t) \\ \dot{\xi}(t) \end{bmatrix} = \begin{bmatrix} A & 0 \\ -C & 0 \end{bmatrix} \begin{bmatrix} x(t) \\ \xi(t) \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} u(t) + \begin{bmatrix} 0 \\ 1 \end{bmatrix} r(t)$$

$$\begin{bmatrix} \dot{x}(t) - \dot{x}(\infty) \\ \dot{\xi}(t) - \dot{\xi}(\infty) \end{bmatrix} = \begin{bmatrix} A & 0 \\ -C & 0 \end{bmatrix} \begin{bmatrix} x(t) - x(\infty) \\ \xi(t) - \xi(\infty) \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} [u(t) - u(\infty)]$$

构造误差状态方程，并判定其能控性：

```
# 构造误差状态方程，并判定系统能控性
Ahat = [A zeros(size(A, 1), 1); -C 0]
Bhat = [B; 0]
r = rank([Ahat Bhat])
```

计算得到 $r = 5$ ，表示系统完全能控，进而可以对系统进行极点配置。

由于闭环系统的阶次为5，需要为其配置5个极点，为保证系统的响应性能，配置其中2个极点为主导极点（方程 $s^2 + 2 * 0.7s + 1 = 0$ 的共轭复根，即阻尼为0.7的理想二阶环节极点），并使其余3个极点远离主导极点(-10、-15、-20)

```
# 计算理想主导极点位置并分配系统所有极点位置
idealPoles = roots([1, 2 * 0.7 * 1, 1])
AllPoles = [idealPoles[1], idealPoles[2], -10, -15, -20]
# 极点配置，计算状态增益K
Khat = place(Ahat, Bhat, AllPoles)
# 状态增益值分解
K = reshape(real(Khat[1:4]), 1, 4)
k1 = -real(Khat[end])
```

```
julia> K = reshape(real(Khat[1:4]), 1, 4)
1x4 Matrix{Float64}:
-982.029 -255.995 -494.898 -419.19

julia> k1 = -real(Khat[end])
-306.1224489795918
```

5.3 基于极点配置法的I型伺服系统设计 – 控制对象无积分器的情况

示例5.3：一维倒立摆系统基于极点配置法的状态反馈控制律设计

③. 控制系统闭环及验证

将 $u = -Kx + k_l \xi$ 带入系统方程形成闭环系统：

$$\begin{bmatrix} \dot{x} \\ \dot{\xi} \end{bmatrix} = \begin{bmatrix} A-BK & Bk_l \\ -C & 0 \end{bmatrix} \begin{bmatrix} x \\ \xi \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} r$$
$$y = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} x \\ \xi \end{bmatrix}$$

闭环系统状态方程

AA = [A-B*K B*k1; -C 0]

BB = [0 0 0 0 1]'

CC = [C 0]

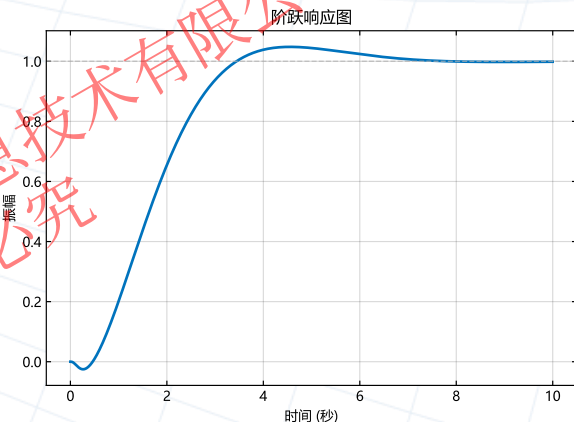
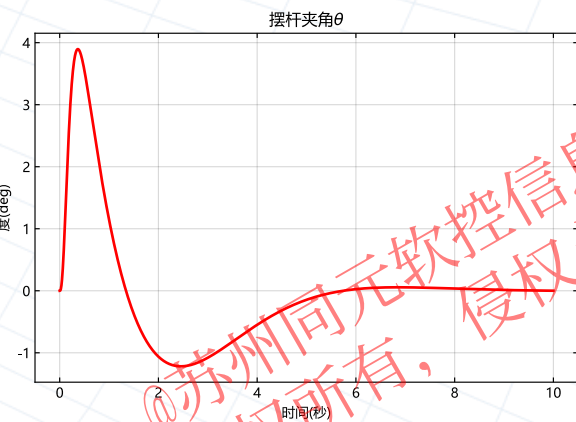
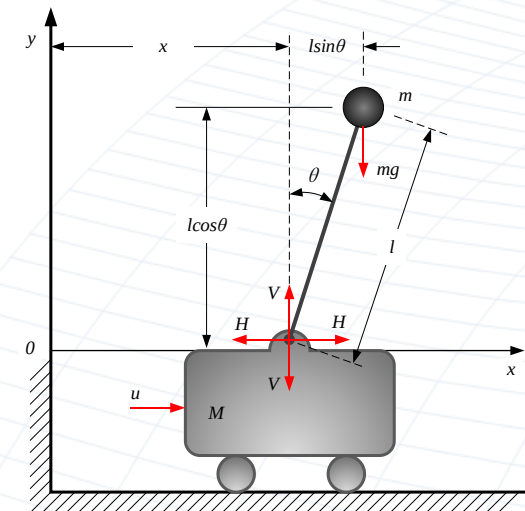
DD = 0

Gclose = ss(AA, BB, CC, DD)

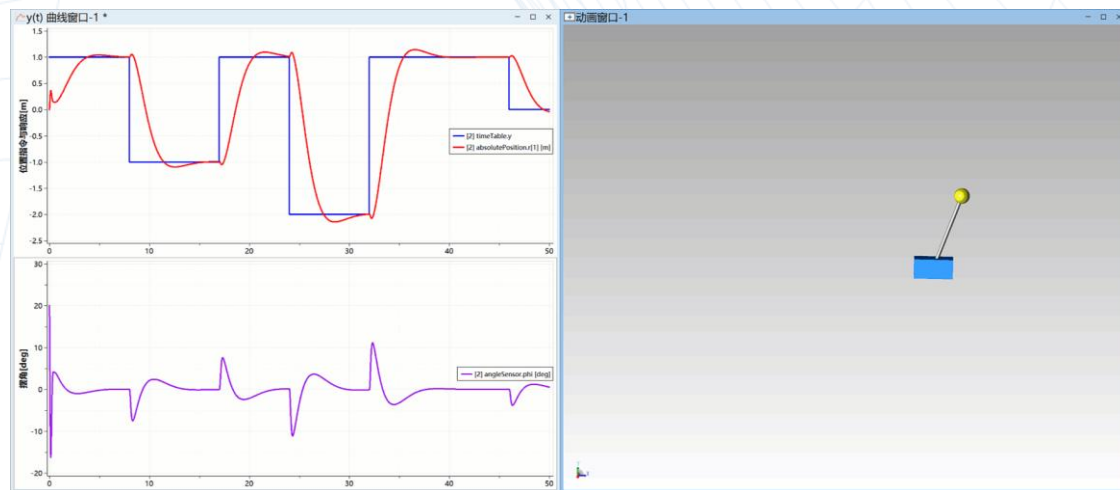
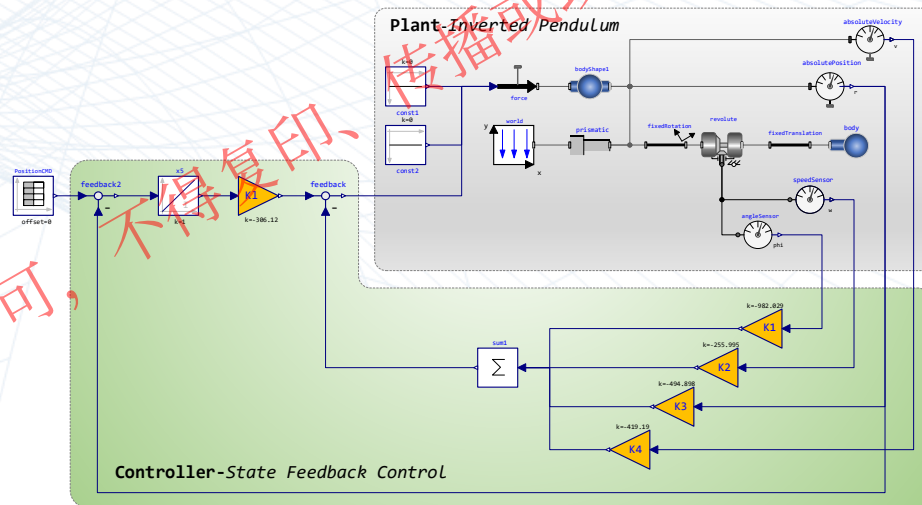
计算并绘制阶跃响应

step(Gclose, 10, linewidth = 2)

grid("on")



④. 在Sysplorer中实现控制系统动态仿真模型并进行验证



5.4 状态反馈控制律设计小结

控制系统的状态空间设计涵盖的内容除了本章介绍的**极点配置**方法，还包括**观测器**、**二次型最佳调节器**系统、**鲁棒控制**等

Syslab目前提供了以下设计函数：

Syslab状态空间设计函数	说明
<code>acker(A, B, p)</code>	用阿克曼法进行极点配置
<code>place(A, B, p)</code>	采用鲁棒极点配置方法计算极点配置
<code>reg = lqg(sys, QXU, QWV)</code> <code>reg = lqg(sys, QXU, QWV, Qi=value)</code>	线性二次高斯（LQG）设计
<code>K, S, P = lqr(sys, Q, R, N)</code> <code>K, S, P = lqr(A, B, Q, R, N)</code>	线性二次调节器(LQR)设计
<code>kalmf, L, P, Mx, Z, My, known, sensors, wEstimates = kalman(sys, Q, R)</code> <code>kalmf, L, P, Mx, Z, My, known, sensors, wEstimates = kalman(sys, Q, R, N = value)</code> <code>___ = kalman(____, estimator=type)</code>	状态估计的卡尔曼滤波设计

具体使用方法及示例，可参考Syslab帮助文档，这里不再赘述

需要说明的是，Syslab控制工具箱函数目前仍在完善过程中，后续版本中将提供更多状态空间设计函数



建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

感谢聆听